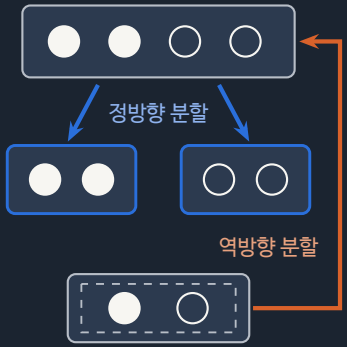
ASWISS

Abstraction Split WIth Sampling by Strategy

학습한 전략의 후회로
추상화를 스스로 나누는 방법



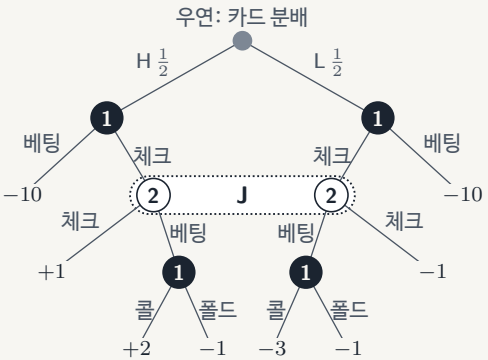
졸업 논문 연구 · 2인 영합 불완전 정보 게임의 적응형 추상화 · 2026년 11월



오늘 발표할 연구는 ASWISS, Abstraction Split WIth Sampling by Strategy입니다.
한 문장으로 말하면, 미리 정한 특징으로 게임을 묶어 두는 대신, 학습한 전략이 "여기는 서로 다르게 두고 싶다"고 말하는 곳만 나누어 가는 추상화 방법입니다.
오른쪽 그림이 발표 전체의 약속입니다. 파란색은 정방향 분할, 주황색은 역방향 분할입니다. 정방향은 한 묶음 안에서 갈리는 것을 그 자리에서 나누고, 역방향은 뒤에서 발견한 차이를 앞 결정으로 거슬러 올라가 나눕니다.

발표는 여덟 가지 질문에 차례로 답하는 순서로 갑니다. 각 슬라이드 오른쪽 위의 표시가 지금 어느 질문에 답하고 있는지 알려 줍니다.

불완전 정보 게임과 정보집합



플레이어 2는 카드를 모른다:
두 상태가 하나의 정보집합 J

게임 트리 · 카드 분배(우연)와 두 플레이어의 행동. 끝의 수는 플레이어 1의 보수(칩)

보는 것 / 못 보는 것 · 플레이어 1(●)은 자기 카드를 본다. 플레이어 2(○)는 행동만 본다

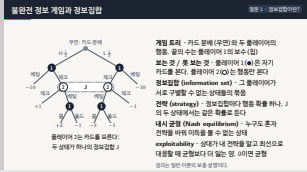
정보집합 (information set) · 그 플레이어가 서로 구별할 수 없는 상태들의 묶음

전략 (strategy) · 정보집합마다 행동 확률 하나. J의 두 상태에서는 같은 확률로 둔다

내시 균형 (Nash equilibrium) · 누구도 혼자 전략을 바꿔 이득을 볼 수 없는 상태

exploitability · 상대가 내 전략을 알고 최선으로 대응할 때 균형보다 더 잃는 양. 0이면 균형

정의는 일반 이론의 보충 설명이다.



먼저 불완전 정보 게임이 무엇인지 이 작은 포커로 보겠습니다. 이 게임은 뒤에서 ASWISS의 동작을 설명할 때도 계속 쓰는 예시입니다.

맨 위 회색 점은 우연, 즉 카드 분배입니다. 플레이어 1이 H 또는 L 카드를 절반씩 받습니다. 검은 원은 플레이어 1, 흰 원은 플레이어 2의 차례이고, 맨 아래 숫자는 플레이어 1이 얻는 칩입니다.

핵심은 플레이어 2가 카드를 보지 못한다는 것입니다. 플레이어 1이 H를 들고 체크했는지 L을 들고 체크했는지 구별할 수 없으므로, 점선으로 묶은 두 상태가 플레이어 2에게는 같은 상황입니다. 이렇게 구별할 수 없는 상태의 묶음을 정보집합이라고 합니다.

전략은 상태마다가 아니라 정보집합마다 정합니다. 그래서 J의 두 상태에서 플레이어 2는 같은 확률로 체크하거나 베팅해야 합니다.

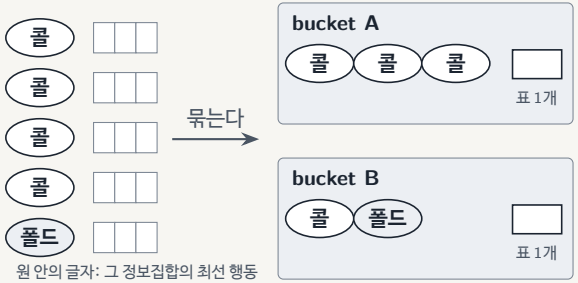
내시 균형은 누구도 혼자 전략을 바꿔서 이득을 볼 수 없는 상태입니다. exploitability는 상대가 내 전략을 알고 최선으로 대응했을 때 균형보다 얼마나 더 잃는지를 재는 값으로, 0이면 균형입니다. 이 값이 이후 실험의 기본 지표입니다.

참고로 이 게임에서 균형은 H면 콜, L이면 폴드, 플레이어 2는 체크이고, 플레이어 1의 값은 0입니다.

추상화: 묶으면 얻는 것과 잃는 것

큰 게임은 정보집합이 너무 많다. 세븐 포커에서는 같은 정보집합이 학습 중 거의 다시 나오지 않아 정보집합마다 표를 둘 수 없다.

정보집합마다 표



원 안의 글자: 그 정보집합의 최선 행동

✓ 이득

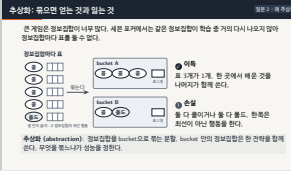
표 3개가 1개. 한 곳에서 배운 것을 나머지가 함께 쓴다.

! 손실

둘 다 콜이거나 둘 다 폴드. 한쪽은 최선이 아닌 행동을 한다.

추상화 (abstraction): 정보집합을 bucket으로 묶는 분할. bucket 안의 정보집합은 한 전략을 함께 쓴다. 무엇을 묶느냐가 성능을 정한다.

추상화: 묶으면 얻는 것과 잃는 것



왜 추상화가 필요한지부터 보겠습니다. 실제 크기의 세븐 포커는 정보집합이 너무 많아서, 학습하는 동안 같은 정보집합을 다시 만나는 일이 거의 없습니다. 정보집합마다 따로 표를 두면 표마다 배울 경험이 없습니다.

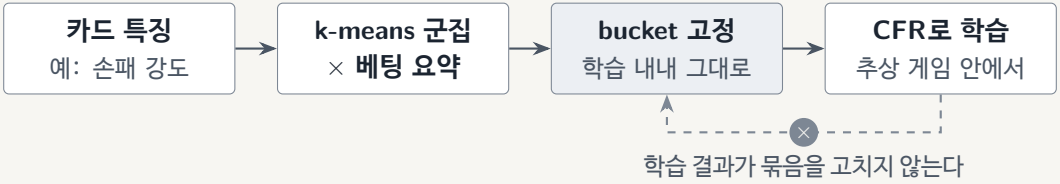
그래서 비슷한 정보집합을 bucket으로 묶고, 묶음 하나가 전략 표 하나를 함께 씁니다. 이것이 추상화입니다.

위의 bucket A처럼 모두 같은 행동을 원하는 정보집합을 묶으면 손해가 없습니다. 표가 줄고, 한 곳에서 배운 것을 나머지가 함께 씁니다.

그러나 아래 bucket B처럼 콜을 원하는 것과 폴드를 원하는 것을 묶으면, 표가 하나뿐이라 둘 중 하나는 최선이 아닌 행동을 하게 됩니다. 이것이 추상화의 손실입니다.

결국 질문은 무엇을 묶고 무엇을 나눌 것인가입니다. 이 결정을 무엇에 근거해 내리느냐가 다음 슬라이드의 주제입니다.

고정 추상화에 빠져 있는 것



❶ 묶는 기준이 해를 보지 않는다

무엇을 합쳐도 되는지는 게임의 해(균형)가 정한다. 카드 특징은 그것을 모른다.

❷ 전략적 동등성의 보장이 없다

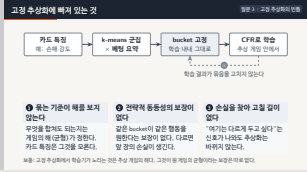
같은 bucket이 같은 행동을 원한다는 보장이 없다. 다르면 앞 장의 손실이 생긴다.

❸ 손실을 찾아 고칠 길이 없다

"여기는 다르게 두고 싶다"는 신호가 나와도 추상화는 바뀌지 않는다.

보충: 고정 추상화에서 학습기가 노리는 것은 추상 게임의 해다. 그것이 원 게임의 균형이라는 보장은 따로 없다.

고정 추상화에 빠져 있는 것



지금까지 큰 게임에서 쓰던 방식은 고정 추상화입니다. 카드 강도 같은 특징을 k-means로 군집하고, 베팅 상황을 요약해서 bucket을 만든 뒤, 그 bucket을 학습 내내 고정해 두고 CFR로 풉니다.

여기에는 세 가지가 빠져 있습니다. 첫째, 묶는 기준이 게임의 해를 보지 않습니다. 무엇을 합쳐도 되는지는 균형에서 같은 행동을 원하느냐로 정해지는데, 카드 특징은 그것을 모릅니다.

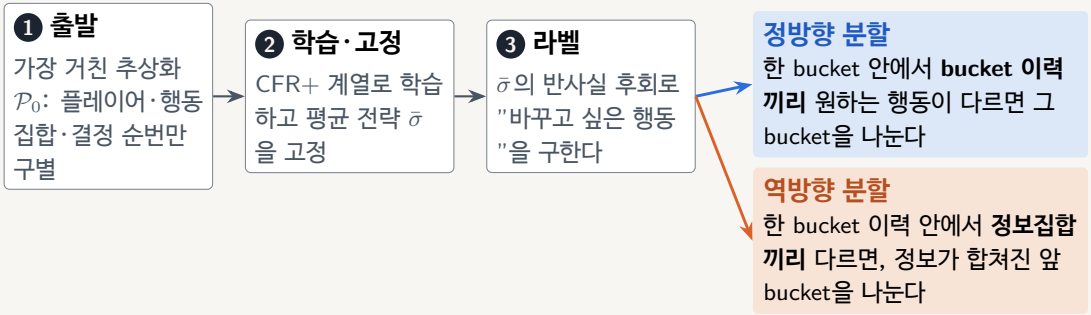
둘째, 그래서 같은 bucket 안의 정보집합들이 같은 행동을 원한다는 보장, 즉 전략적 동등성의 보장이 없습니다. 다르면 앞 장에서 본 손실이 생깁니다.

셋째, 학습 도중에 "여기는 다르게 두고 싶다"는 신호가 나와도 추상화를 고칠 통로가 없습니다. 그림의 끊어진 화살표가 그것입니다.

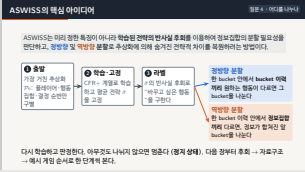
ASWISS는 바로 이 끊어진 화살표를 잇는 방법입니다.

ASWISS의 핵심 아이디어

ASWISS는 미리 정한 특징이 아니라 **학습된 전략의 반사실 후회**를 이용하여 정보집합의 분할 필요성을 판단하고, **정방향** 및 **역방향** **분할**로 추상화에 의해 숨겨진 전략적 차이를 복원하려는 방법이다.



다시 학습하고 판정한다. 아무것도 나뉘지 않으면 멈춘다 (**정지 상태**). 다음 장부터 후회 → 자료구조 → 예시 게임 순서로 한 단계씩 본다.



이것이 이 연구의 핵심 문장입니다. ASWISS는 미리 정한 특징이 아니라, 학습된 전략의 반사실 후회를 이용해서 어떤 정보집합들을 나눠야 하는지 판단하고, 정방향과 역방향 분할로 추상화가 숨긴 전략적 차이를 되살리려는 방법입니다.
절차는 단순합니다. 가장 거친 추상화에서 시작해 학습하고, 그 시점의 평균 전략을 고정합니다. 그 전략 아래에서 후회를 계산해 각 묶음이 "어느 행동으로 바꾸고 싶은지"를 라벨로 붙입니다.
라벨이 갈리는 곳이 분할이 필요한 곳입니다. 갈림이 bucket 안의 이력들 사이에서 보이면 파란색 정방향 분할로 그 bucket을 나누고, 한 이력 안의 정보집합들 사이에 숨어 있으면 주황색 역방향 분할로 정보가 합쳐진 앞 bucket을 나눕니다.

이 문장에 나온 반사실 후회, bucket 이력, 정방향과 역방향을 다음 장부터 하나씩, 같은 예시 게임 위에서 설명하겠습니다.

후회: 그 행동을 했더라면 얼마나 나았을까

예시 게임의 결정 2 (체크 → 상대 베팅 → 콜 또는 폴드). 지금 전략은 "항상 콜"이다.

카드	콜의 값	폴드의 값	폴드의 후회	$\times\frac{1}{2}$
H	$+2$	-1	$-1-(+2)=-3$	$\mathbf{-1.5}$
L	-3	-1	$-1-(-3)=+2$	$\mathbf{+1.0}$

후회 (regret) = 그 행동의 값 − 지금 전략의 값.
양수면 "바꾸고 싶다".

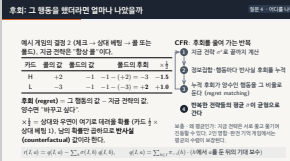
$\times\frac{1}{2}$ = 상대와 우연이 여기로 데려올 확률 (카드 $\frac{1}{2}$ × 상대 베팅 1). 남의 확률만 곱하므로 **반사실 (counterfactual)** 값이라 한다.

CFR: 후회를 줄여 가는 반복

1. 지금 전략 σ^t로 끝까지 계산
2. 정보집합·행동마다 반사실 후회를 누적
3. 누적 후회가 양수인 행동을 그 비율로 둔다 (regret matching)
4. **반복한 전략들의 평균 $\bar{\sigma}$이 균형으로 간다**

보충 · 왜 평균인가: 지금 전략은 서로 쫓고 쫓기며 진동할 수 있다. 2인 영합·완전 기억 게임에서는 평균의 수렴이 보장된다.

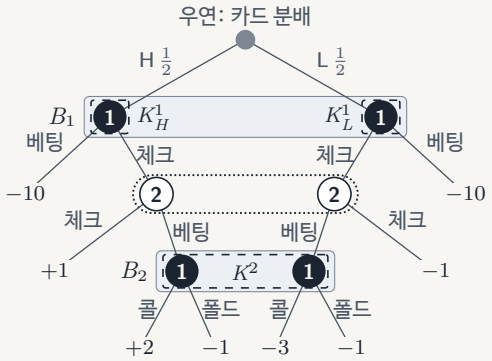
$r(I,a) = q(I,a) - \sum_b \sigma(I,b)\, q(I,b), \qquad q(I,a) = \sum_{h\in I} \pi_{-i}(h) \cdot (h\text{에서 } a\text{를 둔 뒤의 기대 보수})$



분할을 판단하는 재료는 후회입니다. 후회는 "그 행동을 했더라면 얼마나 나았을까"입니다.
예시 게임의 두 번째 결정을 보겠습니다. 플레이어 1이 체크했고 상대가 베팅했습니다. 지금 전략은 항상 콜입니다. H를 들고 있으면 콜이 +2, 폴드가 -1이라 폴드의 후회는 -3입니다. L을 들고 있으면 콜이 -3, 폴드가 -1이라 폴드의 후회는 +2, 즉 폴드로 바꾸고 싶습니다.
여기에 상대와 우연이 이 상태로 데려올 확률을 곱합니다. 카드가 나올 확률 1/2, 상대가 베팅할 확률 1입니다. 내가 여기까지 오려 했다고 가정하고 남의 확률만 곱하기 때문에 반사실 값이라고 부릅니다. 그래서 H는 -1.5, L은 +1.0입니다. 이 두 숫자를 기억해 주십시오. 세 장 뒤에 다시 나옵니다.
CFR은 이 후회를 반복해서 누적하고, 누적 후회가 양수인 행동을 그 크기에 비례해 두는 방식으로 전략을 고칩니다. 지금 전략 자체는 진동할 수 있지만, 반복한 전략들의 평균은 2인 영합 완전 기억 게임에서 균형으로 수렴한다는 것이 알려져 있습니다. 그래서 우리가 내놓는 정책도, 분할 판정의 기준도 평균 전략입니다.

아래 식은 지금 말한 것을 기호로 쓴 것뿐입니다. 자세한 정의는 부록 A1에 있습니다.

bucket과 bucket 이력



출발점 $\mathcal{P}_0$: (플레이어, 행동집합, 자기 결정 순번)이 같은 정보집합을 하나로 묶는 가장 거친 분할

1 정보집합 I · 게임이 정한 구별 단위 (여기서는 플레이어 1의 결정점 하나하나)

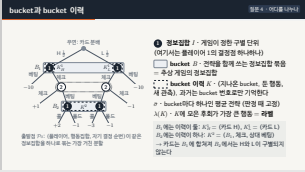
bucket B · 전략을 함께 쓰는 정보집합 묶음 = 추상 게임의 정보집합

bucket 이력 K · (지나온 bucket, 둔 행동, 새 관측). 과거는 bucket 번호로만 기억한다

$\bar{\sigma}$ · bucket마다 하나인 평균 전략 (판정 때 고정)

$\lambda(K)$ · K에 모은 후회가 가장 큰 행동 = **라벨**

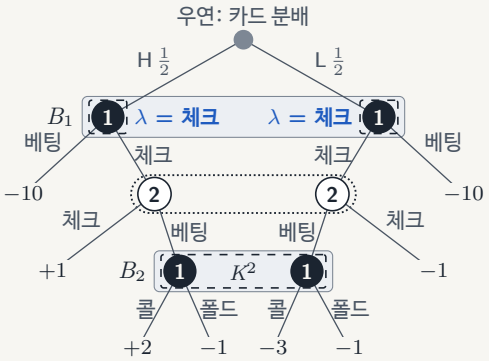
B_1에는 이력이 둘: $K^1_H =$ (카드 H), $K^1_L =$ (카드 L)
B_2에는 이력이 하나: $K^2 =$ (B_1, 체크, 상대 베팅)
→ 카드는 B_1에 합쳐져 B_2에서는 H와 L이 구별되지 않는다



이제 ASWISS가 쓰는 자료구조를 같은 게임 위에서 보겠습니다. 세 가지 기호를 구별해 주십시오.
검은 원 하나하나가 플레이어 1의 정보집합입니다. 회색으로 칠한 상자가 bucket, 즉 전략 표를 함께 쓰는 묶음입니다. 점선 상자가 bucket 이력입니다.
처음에는 가장 거친 분할 P0에서 시작합니다. 같은 플레이어, 같은 행동집합, 같은 결정 순번이면 하나로 묶습니다. 그래서 첫 결정의 H와 L이 B1 하나로, 두 번째 결정의 H와 L이 B2 하나로 묶입니다.
bucket 이력은 그 정보집합에 오기까지 지나온 bucket 번호, 둔 행동, 그리고 새로 관측한 것으로 만든 열쇠입니다. 첫 결정에서는 카드가 새 관측이라 H와 L의 이력이 다릅니다. 그래서 B1 안에는 이력이 두 개 있습니다.
그런데 두 번째 결정에서는 과거를 bucket 번호로만 기억합니다. 지나온 것은 B1, 체크, 상대 베팅으로 H와 L이 똑같습니다. 카드는 이미 B1에 합쳐졌기 때문입니다. 그래서 B2 안에는 이력이 하나뿐이고, 그 안에 H와 L 두 정보집합이 들어 있습니다.

정리하면, 전략은 bucket 단위로 공유하고, 라벨은 bucket 이력 단위로 모읍니다. 한 bucket 안에 서로 다른 행동을 원하는 이력이 있을 수도 있고, 한 이력 안에 서로 다른 행동을 원하는 정보집합이 있을 수도 있습니다. 앞의 것을 정방향이, 뒤의 것을 역방향이 다룹니다.

정방향 분할: 이력끼리 라벨이 갈리면 나눈다

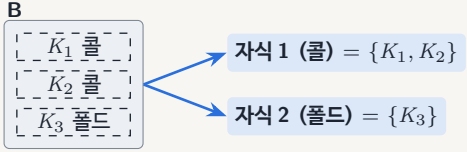


B_1 의 두 이력 모두 "체크" → 유지 (옳다)

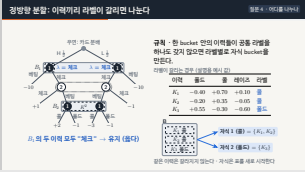
규칙 · 한 bucket 안의 이력들이 공통 라벨을 하나도 갖지 않으면 라벨별로 자식 bucket을 만든다.

라벨이 갈리는 경우 (설명용 예시 값)

이력	폴드	콜	레이즈	라벨
K_1	-0.40	$+0.70$	$+0.10$	콜
K_2	-0.20	$+0.35$	-0.05	콜
K_3	$+0.55$	-0.30	-0.60	폴드



같은 이력은 갈라지지 않는다 · 자식은 표를 새로 시작한다



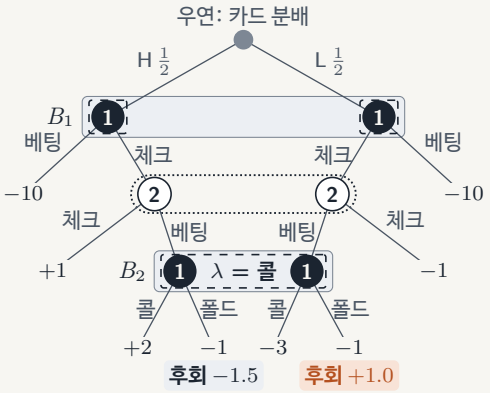
첫 번째 분할 규칙인 정방향 분할입니다. 평균 전략을 고정하고, bucket 이력마다 후회를 모아 라벨, 즉 가장 바꾸고 싶은 행동을 붙입니다.

오른쪽 표는 설명용 예시입니다. 한 bucket B 안에 이력이 세 개 있는데 K1과 K2는 콜을, K3는 폴드를 원합니다. 셋이 함께 원하는 라벨이 없으므로 B를 라벨별로 나눕니다. 콜을 원하는 K1, K2가 한 자식으로, 폴드를 원하는 K3가 다른 자식으로 갑니다. 두 가지를 지킵니다. 같은 이력은 절대 둘로 갈라지지 않고 늘 한 bucket에 있습니다. 그리고 새 자식 bucket은 표를 새로 시작합니다.

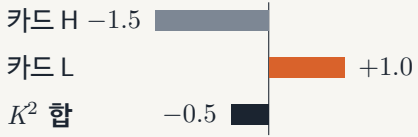
왼쪽 예시 게임에 적용해 보겠습니다. 첫 결정의 B1에는 이력이 H와 L 두 개입니다. 베팅하면 -10이라 두 이력 모두 체크가 최선이고, 라벨이 같으므로 나누지 않습니다. 첫 결정만 보면 이것은 옳은 판단입니다.

문제는 두 번째 결정입니다. 다음 장에서 보겠습니다.

정방향 분할만으로는 멈출 수 있다 (명제 H4)



결정 2에서 폴드의 반사실 후회 (앞의 표)

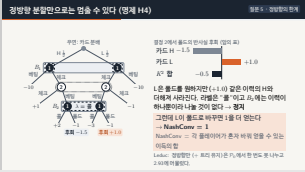


L은 폴드를 원하지만 ($+1.0$) 같은 이력의 H와 더해져 사라진다. 라벨은 "콜"이고 B_2에는 이력이 하나뿐이라 나눌 것이 없다 → **정지**

그런데 L이 폴드로 바꾸면 1을 더 얻는다
→ **NashConv = 1**

NashConv = 각 플레이어가 혼자 바꿔 얻을 수 있는 이득의 합

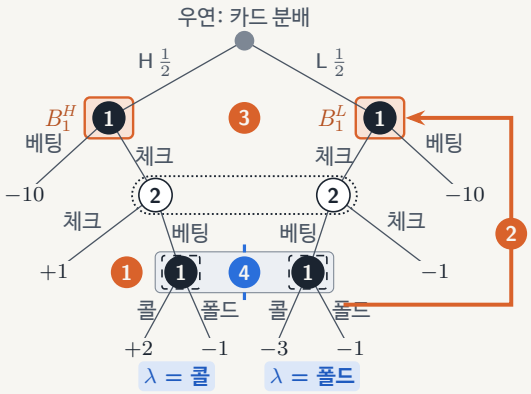
Leduc: 정방향만 (+ 트리 유지)은 $\mathcal{P}_0$에서 한 번도 못 나누고 2.93에 머물렀다.



정방향 분할만으로는 멈출 수 있다는 것이 명제 H4입니다. 같은 게임의 두 번째 결정을 보겠습니다.
앞에서 계산한 폴드의 반사실 후회가 여기 다시 나옵니다. H는 -1.5, L은 +1.0입니다. L은 폴드로 바꾸고 싶어 합니다.
그런데 두 번째 결정에서는 H와 L이 같은 bucket 이력 K2에 들어 있습니다. 라벨은 이력 단위로 후회를 더해서 정하므로 -1.5 더하기 +1.0, 즉 -0.5가 되고, 라벨은 콜입니다. L의 양의 후회가 H의 음의 후회에 상쇄되어 보이지 않습니다.
B2에는 이력이 하나뿐이니 정방향 분할은 나눌 것이 없습니다. 첫 결정의 B1은 앞 장에서 본 것처럼 나눌 이유가 없습니다.
그래서 알고리즘은 여기서 멈춥니다.
하지만 이 상태는 균형이 아닙니다. L을 든 플레이어 1이 폴드로 바꾸면 1을 더 얻으므로 NashConv가 1입니다. 정확한 학습기와 정확한 라벨을 써도 이렇게 멈추는 게임이 있다는 것이 명제 H4입니다.

이것은 작은 예시만의 문제가 아니었습니다. Leduc 홀덤에서 정방향만 쓰고 트리 구조를 유지하면 가장 거친 분할에서 한 번도 나누지 못하고 exploitability 2.93에 머물렀습니다.

역방향 분할: 정보가 합쳐진 앞 bucket을 나눈다



1 진단

이력 K^2 안의 정보집합끼리 라벨이 갈린다: H는 콜, L은 폴드 ($H_K > 0$)

2 거슬러 오르기

이력이 달랐는데 같은 bucket에 묶인 앞 결정 → 결정 1의 B_1 (보조정리 H3)

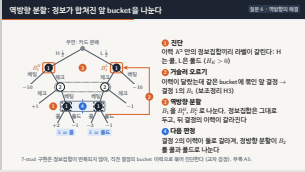
3 역방향 분할

B_1을 B_1^H, B_1^L로 나눈다. 정보집합은 그대로 두고, 뒤 결정의 이력이 갈라진다

4 다음 판정

결정 2의 이력이 둘로 갈라져, 정방향 분할이 B_2를 콜과 폴드로 나눈다

7-stud 구현은 정보집합이 반복되지 않아, 직전 결정의 bucket 이력으로 묶어 진단한다 (교차 검정). 부록 A5.



이 문제를 푸는 것이 역방향 분할입니다. 네 단계로 보겠습니다.

1번, 진단입니다. 라벨을 이력 단위로만 보지 않고, 이력 안의 정보집합별 후회도 봅니다. K2 안에서 H는 콜, L은 폴드를 원하니 서로 공통 라벨이 없습니다. 인증식으로 말하면 이력 안 상쇄 H_K가 0보다 큽니다.

2번, 거슬러 오르기입니다. 두 정보집합의 과거를 거꾸로 따라가면서, 이력은 달랐는데 같은 bucket에 들어간 결정을 찾습니다. 이 게임에서는 첫 결정입니다. 첫 결정에서는 카드가 달라 이력이 달랐는데 B1이라는 같은 bucket에 묶였고, 그 뒤로는 구별되지 않았습니다. 이런 지점이 늘 존재한다는 것이 보조정리 H3입니다. bucket 이력에 직전 bucket이 들어 있기 때문에 이 지점을 찾을 수 있습니다.

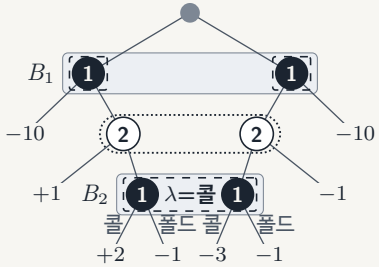
3번, 그 bucket B_1을 카드별로 나눕니다. 주황색 상자가 그것입니다. 정보집합을 옮기는 것이 아니라 앞 bucket만 쪼갭니다.

4번, 그러면 두 번째 결정의 이력이 B_1-H를 지나온 것과 B_1-L을 지나온 것으로 갈라집니다. 이제 H의 이력은 콜, L의 이력은 폴드를 원하므로, 다음 판정에서 정방향 분할이 B_2를 나눌 수 있습니다. 파란 점선이 나뉠 자리입니다.

7-stud처럼 큰 게임에서는 정보집합이 반복되지 않아 정보집합별 후회를 쓸 수 없으므로, 직전 결정의 bucket 이력으로 묶어서 같은 진단을 하는 약한 판정을 씁니다. 그 차이는 뒤에서 한계로 다시 말씀드리겠습니다.

같은 게임, 두 결과: 정방향만 vs 역방향 포함

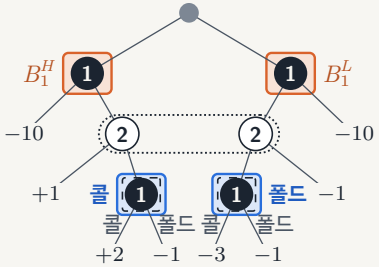
정방향만



정지 · NashConv = 1

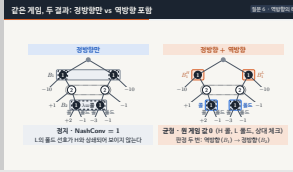
L의 폴드 선호가 H와 상쇄되어 보이지 않는다

정방향 + 역방향



균형 · 원 게임 값 0 (H 콜, L 폴드, 상대 체크)

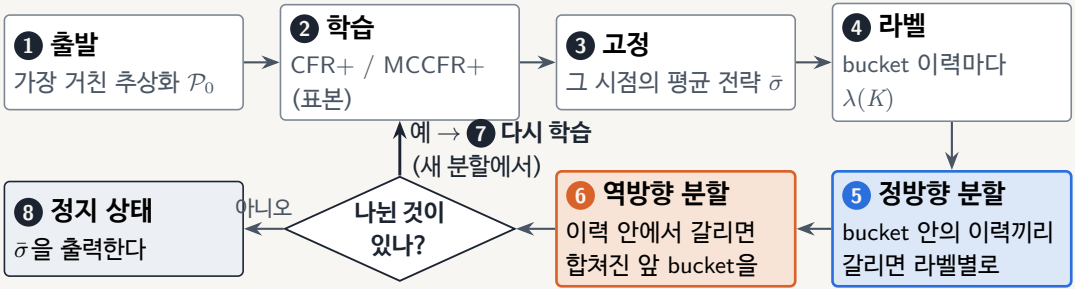
판정 두 번: 역방향(B_1) → 정방향(B_2)



같은 게임에서 두 방법이 도착하는 곳을 나란히 놓았습니다.
왼쪽은 정방향만 쓴 경우입니다. B1과 B2가 처음 그대로이고, B2 안의 이력은 하나라 라벨이 콜입니다. 여기서 멈추고, L이 폴드로 바꾸면 1을 더 얻으므로 NashConv는 1입니다. 플레이어 1의 값은 -0.5입니다.
오른쪽은 역방향 분할을 더한 경우입니다. 첫 판정에서 역방향 분할이 B1을 카드별로 나누고, 다음 판정에서 정방향 분할이 B2를 콜과 폴드로 나눕니다. 판정 두 번 뒤에 플레이어 1은 H면 콜, L이면 폴드를 하게 되고, 그러면 플레이어 2는 체크하는 것이 낫습니다. 이것이 원 게임의 균형이고 플레이어 1의 값은 0입니다.
중요한 점은 오른쪽도 정보집합 하나하나를 다 나눈 것이 아니라는 점입니다. 필요한 곳, 즉 숨은 차이가 있던 두 bucket만 나눴습니다.

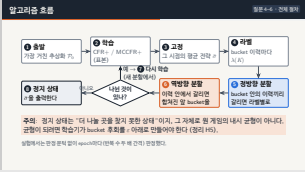
이 동작이 일반적으로 성립한다는 것을 보이는 것이 다음 이론 부분입니다.

알고리즘 흐름



주의: 정지 상태는 "더 나눌 곳을 찾지 못한 상태"이지, 그 자체로 원 게임의 내시 균형이 아니다. 균형이 되려면 학습기가 bucket 후회를 ε 아래로 만들어야 한다 (정리 H5).

실험에서는 판정 문턱 없이 epoch마다 (반복 수 두 배 간격) 판정했다.



지금까지의 내용을 하나의 절차로 모으면 이렇습니다.
1번, 가장 거친 추상화 P0에서 출발합니다. 2번, CFR+로 학습합니다. 큰 게임에서는 표본을 쓰는 MCCFR+입니다. 3번, 그 시점의 평균 전략을 고정합니다. 4번, 고정한 전략 아래에서 bucket 이력마다 라벨을 계산합니다.
5번, 한 bucket 안의 이력들이 공통 라벨을 갖지 않으면 정방향 분할로 나눕니다. 6번, 한 이력 안의 정보집합들이 갈리면 역방향 분할로 정보가 합쳐진 앞 bucket을 나눕니다.
나뉜 것이 있으면 7번, 새 분할에서 다시 학습하고 2번으로 돌아갑니다. 아무것도 나뉘지 않으면 8번, 멈추고 평균 전략을 내놓습니다. 이것을 정지 상태라고 부릅니다.

여기서 꼭 강조하고 싶은 것은, 정지 상태가 자동으로 원 게임의 내시 균형은 아니라는 점입니다. 정지 상태는 더 나눌 곳을 찾지 못했다는 뜻입니다. 균형이라고 말하려면 학습기가 bucket 후회를 충분히 작게 만들어야 한다는 조건이 필요하고, 그것이 정리 H5의 가정입니다. 실험에서는 문턱 없이 epoch마다 판정했습니다.

의사코드와 정의

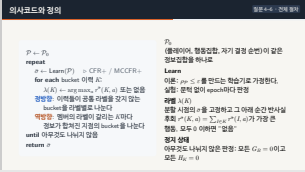
$\mathcal{P} \leftarrow \mathcal{P}_0$
repeat
  $\bar{\sigma} \leftarrow \text{Learn}(\mathcal{P})$ ▷ CFR+ / MCCFR+
  for each bucket 이력 K:
    $\lambda(K) \leftarrow \arg\max_a r^{\bar{\sigma}}(K, a)$ 또는 없음
  정방향: 이력들이 공통 라벨을 갖지 않는 bucket을 라벨별로 나눈다
  역방향: 멤버의 라벨이 갈리는 K마다 정보가 합쳐진 지점의 bucket을 나눈다
until 아무것도 나뉘지 않음
return $\bar{\sigma}$

$\mathcal{P}_0$
(플레이어, 행동집합, 자기 결정 순번)이 같은 정보집합을 하나로

Learn
이론: $\rho_{\mathcal{P}} \le \varepsilon$를 만드는 학습기로 가정한다.
실험: 문턱 없이 epoch마다 판정

라벨 $\lambda(K)$
분할 시점의 $\bar{\sigma}$을 고정하고 그 아래 순간 반사실 후회 $r^{\bar{\sigma}}(K, a) = \sum_{I \in K} r^{\bar{\sigma}}(I, a)$가 가장 큰 행동. 모두 0 이하면 "없음"

정지 상태
아무것도 나뉘지 않은 판정: 모든 $G_B = 0$이고 모든 $H_K = 0$



같은 절차를 의사코드로 쓰면 이렇습니다. 논문에서 증명하는 기본 형태입니다.
P를 P0로 두고, 반복마다 현재 분할에서 학습해 평균 전략을 얻습니다. 모든 bucket 이력에 라벨을 붙이고, 정방향과 역방향 분할을 합니다. 아무것도 나뉘지 않으면 멈추고 평균 전략을 돌려줍니다.
오른쪽은 각 줄의 정의입니다. 이론에서는 Learn을, bucket 후회를 ε 아래로 만드는 학습기로 가정합니다. 실험에서는 그런 문턱을 두지 않고 epoch마다 판정했습니다. 이 차이는 마지막 실험에서 다시 다룹니다.
라벨은 분할 시점의 평균 전략을 고정하고, 그 아래의 순간 반사실 후회를 이력 안에서 더해 가장 큰 행동입니다. 모두 0 이하면 지금 전략이 최선이라는 뜻에서 "없음"입니다.
정지 상태는 아무것도 나뉘지 않은 판정이고, 다음 장의 인증식에서 두 상쇄 항 G와 H가 모두 0인 상태와 같습니다.

분할은 정보집합을 옮기지 않고 bucket만 쪼개므로, 분할 횟수는 정보집합 수를 넘지 못합니다. 이것이 유한 종료입니다.

인증 부등식: 세 항이 각각 무엇을 막는가

정리 H1 (어떤 lifting된 전략 σ에도 성립)

$$\mathrm{NashConv}(\sigma) \;\le\; \rho_{\mathcal{P}}(\sigma) \;+\; \sum_B G_B \;+\; \sum_K H_K$$

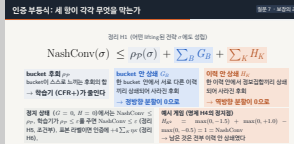
bucket 후회 $\rho_{\mathcal{P}}$
bucket이 스스로 느끼는 후회의 합
→ **학습기 (CFR+)가 줄인다**

bucket 안 상쇄 G_B
한 bucket 안에서 서로 다른 이력끼리 상쇄되어 사라진 후회
→ **정방향 분할이 0으로**

이력 안 상쇄 H_K
한 이력 안에서 정보집합끼리 상쇄되어 사라진 후회
→ **역방향 분할이 0으로**

정지 상태 ($G = 0$, $H = 0$)에서는 $\mathrm{NashConv} \le \rho_{\mathcal{P}}$. 학습기가 $\rho_{\mathcal{P}} \le \varepsilon$를 주면 $\mathrm{NashConv} \le \varepsilon$ (정리 H5, 조건부). 표본 라벨이면 인증에 $+4\sum_K \eta_K$ (정리 H6).

예시 게임 (명제 H4의 정지점)
$H_{K^2} = \max(0, -1.5) + \max(0, +1.0) - \max(0, -0.5) = 1 = \mathrm{NashConv}$
→ 남은 것은 전부 이력 안 상쇄였다



이론의 중심은 이 인증 부등식, 정리 H1입니다. 원 게임에서의 NashConv가 세 항의 합으로 위에서 막힙니다. 어떤 lifting된 전략, 즉 bucket마다 하나의 전략을 쓰는 어떤 전략에도 성립합니다.

첫째 항 rho_P는 bucket 후회입니다. bucket을 하나의 정보집합처럼 보았을 때 느끼는 후회로, 학습기가 줄이는 몫입니다.

둘째 항 G_B는 한 bucket 안에서 서로 다른 이력끼리 상쇄되어 사라진 후회입니다. 라벨이 갈리는 bucket을 나누는 정방향 분할이 이 항을 0으로 만듭니다.

셋째 항 H_K는 한 이력 안에서 정보집합끼리 상쇄되어 사라진 후회입니다. 라벨 판정으로는 볼 수 없고, 역방향 분할이 이 항을 0으로 만듭니다. 학습기, 정방향, 역방향이 세 항을 하나씩 맡는 구조입니다.

앞의 예시 게임에 넣어 보면, 정지점에서 H_K는 0 더하기 1 빼기 0으로 1이고, 실제 NashConv도 1입니다. 남은 오차가 전부 이력 안 상쇄였다는 뜻입니다.

정지 상태에서는 두 상쇄 항이 0이므로 NashConv는 bucket 후회 이하가 됩니다. 그래서 학습기가 bucket 후회를 ε 아래로 만들어 준다면 원 게임의 NashConv도 ε 이하라는 것이 정리 H5입니다. 다만 이것은 조건부입니다. 표본으로 라벨을 추정하면 추정 오차 항이 더해집니다.

무엇이 증명되었고, 무엇이 조건부인가

증명 · 부등식

H1 인증: $\mathrm{NashConv} \le \rho_{\mathcal{P}} + \sum G_B + \sum H_K$ · **H6 표본 판정**: 좋은 사건 위에서 거짓 분할이 없고 인증에 $+4\sum_K \eta_K$

증명 · 유한 종료

H2: 분할은 많아야 $N - |\mathcal{P}_0|$번 ($N=$ 정보집합 수). 라벨이 추정이어도 같다

증명 · 반례와 동치

H4: 정방향만으로 NashConv $= 1$에서 멈추는 게임이 있다 · **H7**: 모든 bucket의 부모(직전 bucket, 행동)가 하나 $\Leftrightarrow$ 추상화의 완전 기억

조건부 정리

H5: 학습기가 $\rho_{\mathcal{P}} \le \varepsilon$를 주고 라벨이 정확하면, 정방향 + 역방향은 $\mathrm{NashConv} \le \varepsilon$에서 멈춘다

실험적 수렴

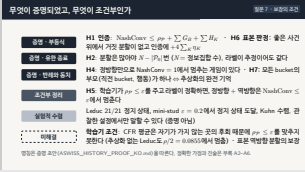
Leduc 21/21 정지 상태, mini-stud $\varepsilon = 0.2$에서 정지 상태 도달, Kuhn 수렴. 관찰한 설정에서만 말할 수 있다 (증명 아님)

미해결

학습기 조건: CFR 평균은 자기가 가지 않는 곳의 후회 때문에 $\rho_{\mathcal{P}} \le \varepsilon$를 맞추지 못한다 (추상화 없는 Leduc도 $\rho/2 = 0.0855$에서 멈춤) · 표본 역방향 분할의 보장

명칭은 증명 초안(ASWISS_HISTORY_PROOF_KO.md)을 따른다. 정확한 가정과 진술은 부록 A2–A6.

└무엇이 증명되었고, 무엇이 조건부인가



이론 결과를 상태별로 정리했습니다. 무엇이 증명이고 무엇이 조건부인지 구별하는 것이 중요합니다.
증명된 것은 세 줄입니다. 인증 부등식 H1과, 라벨을 표본으로 추정할 때의 버전 H6. 분할 횟수가 정보집합 수를 넘지 못한다는 유한 종료 H2. 정방향만으로는 NashConv 1에서 멈추는 게임이 있다는 반례 H4, 그리고 모든 bucket이 부모를 하나만 가지는 트리 구조가 추상화의 완전 기억과 같다는 H7입니다.
원 게임의 균형에 대한 보장은 H5 하나이고, 이것은 조건부입니다. 학습기가 bucket 후회를 ε 아래로 만들고 라벨이 정확하다면, 정방향과 역방향을 함께 쓴 ASWISS는 NashConv ε 이하에서 멈춥니다.
실험적 수렴은 증명이 아니라 관찰입니다. Leduc의 21개 출발점 모두, mini-stud의 ε 0.2 실행이 정지 상태에 도달했다는 것은 그 설정에서의 사실입니다.

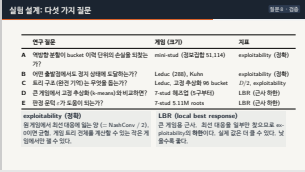
미해결로 남은 것이 학습기 조건입니다. 표준 CFR의 평균 전략은 자기가 가지 않는 곳의 전략을 거의 고치지 않아서, 추상화가 전혀 없는 Leduc에서도 bucket 후회의 절반이 0.0855에서 멈춥니다. 그때 exploitability는 이미 0.000067로 아주 작은데도 그렇습니다. 그래서 H5의 가정을 CFR이 만족한다고 말할 수 없습니다. 또 표본으로 판정하는 역방향 분할에 대한 보장도 아직 없습니다.

실험 설계: 다섯 가지 질문

	연구 질문	게임 (크기)	지표
A	역방향 분할이 bucket 이력 단위의 손실을 되찾는가?	mini-stud (정보집합 51,114)	exploitability (정확)
B	어떤 출발점에서도 정지 상태에 도달하는가?	Leduc (288), Kuhn	exploitability (정확)
C	트리 구조(완전 기억)는 무엇을 돕는가?	Leduc, 고정 추상화 96 bucket	$D/2$, exploitability
D	큰 게임에서 고정 추상화(k-means)와 비교하면?	7-stud 헤즈업 (5구부터)	LBR (근사 하한)
E	판정 문턱 ε가 도움이 되는가?	7-stud 5.11M roots	LBR (근사 하한)

exploitability (정확)
원 게임에서 최선 대응에 잃는 양 ($=$ NashConv / 2). 0이면 균형. 게임 트리 전체를 계산할 수 있는 작은 게임에서만 잴 수 있다.

LBR (local best response)
큰 게임용 근사. 최선 대응을 일부만 찾으므로 exploitability의 **하한**이다. 실제 값은 더 클 수 있다. 낮을수록 좋다.


실험 설계: 다섯 가지 질문

	연구 질문	게임 (크기)	지표
A	역방향 분할이 bucket 이력 단위의 손실을 회복은가?	mini-stud (정보집합 51,114)	exploitability (경확)
B	어떤 출발점에서도 정지 상태에 도달하는가?	Leduc (288), Kuhn	exploitability (경확)
C	트리 구조(왕은 기억)는 무엇을 중는가?	Leduc, 고정 추상화 96 bucket	Π^*, exploitability
D	큰 게임에서 고정 추상화 (k-means)와비교하면?	7-stud 제조업 (1-구부터)	LBR (근사 하한)
E	한편 공식 ε가 도움이 되는가?	7-stud 5.11M roots	LBR (근사 하한)

exploitability (경확)
동 게임에서 최선 대응에 얼는 양 (— NashConv / 2). 00만 근방. 게임 트리 전체를 계산할 수 있는 작은 게임에서만 볼 수 있다.

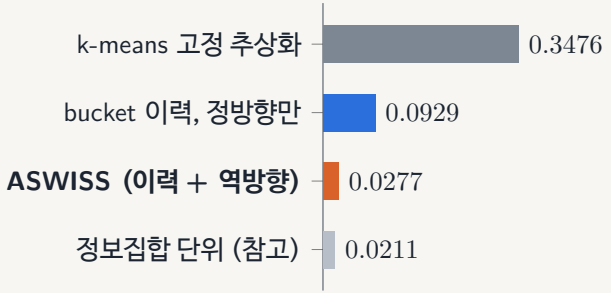
LBR (local best response)
큰 게임을 근사. 최선 대응을 일부만 찾으므로 exploitability의 하한이다. 실제 값은 더 클 수 있다. 낮을수록 좋다.

실험은 다섯 가지 질문에 답하도록 짰습니다.
A는 mini-stud에서, 역방향 분할이 bucket 이력을 쓰면서 생긴 손실을 되찾는지 봅니다. B는 Leduc과 Kuhn에서, 가장 거친 분할이든 무작위 추상화든 어떤 출발점에서도 정지 상태에 도달하는지 봅니다. C는 트리 구조, 즉 추상화의 완전 기억이 무엇을 돕는지 봅니다.
D와 E는 실제 크기의 7-stud입니다. D는 고정 k-means 추상화와의 비교, E는 이론의 학습기 조건을 판정 문턱으로 흉내 냈을 때 도움이 되는지입니다.

지표를 구별해 주십시오. A부터 C까지는 게임 트리 전체를 계산할 수 있어서 정확한 exploitability를 씁니다. 7-stud는 그럴 수 없어서 LBR을 씁니다. LBR은 최선 대응을 일부만 찾는 근사라서 exploitability의 하한입니다. 실제 exploitability는 더 클 수 있으니, LBR 숫자를 exploitability와 같은 것으로 읽으면 안 됩니다.

A. mini-stud: 역방향 분할이 손실을 되찾았다

질문 A · 역방향 분할이 bucket 이력 단위의 손실을 되찾는가?



정보집합 51,114개 · 4,096만 반복 · seed 3개 평균 · 평균 정책의 정확한 exploitability (낮을수록 좋음) · 비교군마다 bucket 수가 다르다 (부록 A8)

3.4배

역방향 분할을 더하자 0.0929 → 0.0277

정지 상태 도달

$\varepsilon = 0.2$ 비교군: 2,900만–3,050만 반복에 $G = 0$, $H = 0$. 실제 0.034–0.038 $\leq$ 인증 0.19–0.20

0.0211

정보집합 단위 라벨이 조금 더 낮지만, 큰 게임에서는 정보집합이 반복되지 않아 쓸 수 없다

A. mini-stud: 역방향 분할이 손실을 되찾았다



첫 번째 질문입니다. mini-stud는 정보집합이 51,114개라 모두 열거할 수 있어서 정확한 exploitability를 잴 수 있습니다. 4,096만 반복, seed 세 개의 평균이고 낮을수록 좋습니다.

맨 위 회색이 고정 k-means 추상화로 0.3476입니다. 파란색이 bucket 이력에 정방향만 쓴 것으로 0.0929, 주황색이 역방향 분할까지 넣은 ASWISS로 0.0277입니다. 역방향 분할을 더하자 3.4배 낮아졌습니다. 앞의 예시 게임에서 본 이력 안 상쇄가 실제 게임에서도 손실의 큰 부분이었다는 뜻입니다.

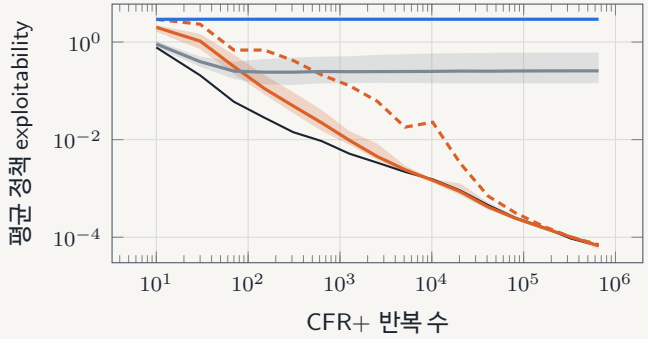
맨 아래 연한 회색은 정보집합 단위로 라벨을 매긴 비교용 변형입니다. 조금 더 낮지만, 큰 게임에서는 같은 정보집합이 거의 다시 나오지 않아 쓸 수 없는 방법입니다.

오른쪽 가운데는 이론의 정지 상태가 실제로 나오는지 본 결과입니다. 학습기 조건을 ε 0.2의 문턱으로 강제한 비교군에서, 2,900만에서 3,050만 반복 사이에 G와 H가 모두 0인 판정이 나왔고, 그때 실제 값 0.034에서 0.038이 인증값 0.19에서 0.20 아래에 있었습니다.

주의할 점은 비교군마다 최종 bucket 수가 다르다는 것입니다. 같은 bucket 수의 비교가 아니므로 부록 A8에 수를 적어 두었습니다.

B. Leduc: 어떤 출발점에서도 정지 상태에 도달

질문 B · 가장 거친 분할이든 무작위 추상화든, 정지 상태와 낮은 exploitability에 도달하는가?



추상화 없는 CFR+
ASWISS, 무작위 출발 20개 (중앙값)
ASWISS, 가장 거친 $\mathcal{P}_0$ 에서
같은 bucket 수의 k-means (중앙값)
정방향만 + 트리 유지, $\mathcal{P}_0$ 에서

21 / 21

$\mathcal{P}_0$와 무작위 출발 20개 모두 정지 상태 ($G = 0$, $H = 0$)

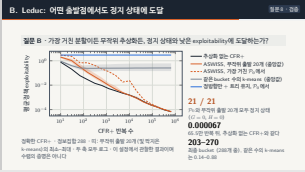
0.000067

65.5만 반복 뒤. 추상화 없는 CFR+와 같다

203–270

최종 bucket (288개 중). 같은 수의 k-means는 0.14–0.88

정확한 CFR+ · 정보집합 288 · 띠: 무작위 출발 20개 (및 짝지은 k-means)의 최소–최대 · 두 축 모두 로그 · 이 설정에서 관찰한 결과이며 수렴의 증명은 아니다



두 번째 질문은 출발점과 상관없이 정지 상태에 도달하느냐입니다. Leduc 홀덤은 정보집합이 288개라 정확한 CFR+와 정확한 exploitability를 쏠 수 있습니다. 두 축 모두 로그 축입니다.

검은 선이 추상화 없는 CFR+, 즉 이 학습량에서 도달할 수 있는 기준입니다. 주황색 실선이 무작위 추상화 20개에서 출발한 ASWISS의 중앙값, 주황색 파선이 가장 거친 P0에서 출발한 ASWISS입니다. 처음에는 거친 추상화 때문에 높지만, 분할이 진행되면서 검은 선에 붙습니다.

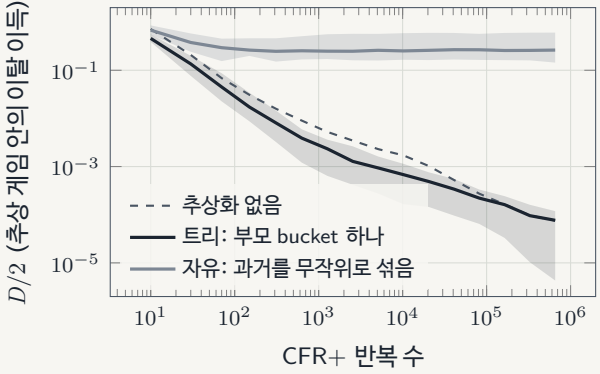
회색은 ASWISS와 같은 bucket 수로 만든 k-means 고정 추상화입니다. 0.1 아래로 내려가지 않습니다. 파란 선은 정방향만 쓰고 트리 구조를 유지한 경우로, 앞에서 말한 대로 2.93에서 움직이지 않습니다.

정리하면 21개 출발점 모두 정지 상태에 도달했고, 65.5만 반복 뒤 exploitability 0.000067로 추상화 없는 CFR+와 같았습니다. 최종 bucket은 288개 중 203에서 270개로, 정보집합을 다 나누지 않고도 같은 값에 왔습니다. 같은 수의 k-means는 0.14에서 0.88이었습니다.

다만 이것은 이 게임, 이 설정에서 관찰한 결과입니다. 수렴을 증명한 것은 아닙니다.

C. 트리 구조와 역방향 분할의 역할

질문 C · 트리 구조(각 bucket의 부모가 하나 = 완전 기억)는 무엇을 돕고, 무엇을 돕지 않는가?



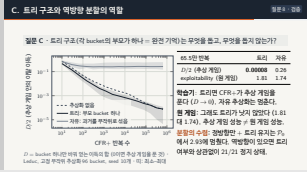
D = bucket 하나만 바꿔 얻는 이득의 합 (0이면 추상 게임을 푼 것) · Leduc, 고정 무작위 추상화 96 bucket, seed 10개 · 띠: 최소–최대

65.5만 반복	트리	자유
$D/2$ (추상 게임)	**0.00008**	0.26
exploitability (원 게임)	1.81	1.74

학습기: 트리면 CFR+가 추상 게임을 푼다 ($D \to 0$). 자유 추상화는 멈춘다.

원 게임: 그래도 트리가 낫지 않았다 (1.81 대 1.74). 추상 게임 성능 $\neq$ 원 게임 성능.

분할의 수렴: 정방향만 + 트리 유지는 $\mathcal{P}_0$에서 2.93에 멈췄다. 역방향이 있으면 트리 여부와 상관없이 21/21 정지 상태.



세 번째 질문은 트리 구조의 역할입니다. 트리 구조란 각 bucket이 부모, 즉 직전 bucket과 그때 둔 행동을 하나만 갖는 추상화이고, 이것이 추상화의 완전 기억과 같다는 것이 H7입니다.

그래프의 세로축 D는 추상 게임 안에서 bucket 하나만 바꿔 얻을 수 있는 이득의 합입니다. 0이면 추상 게임을 푼 것입니다. 검은 선이 트리 추상화로, 추상화 없는 경우처럼 0으로 내려갑니다. 회색은 과거를 무작위로 섞은 자유 추상화로, 0.26 근처에서 멈춥니다. 트리 구조는 학습기를 확실히 돕습니다.

그런데 원 게임 exploitability는 트리가 1.81, 자유가 1.74로 트리가 낫지 않았습니다. 추상 게임을 잘 푸는 것과 원 게임에서 좋은 전략인 것은 다른 이야기입니다.

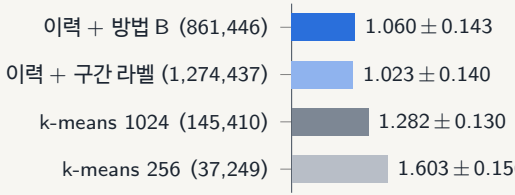
그리고 분할 과정의 수렴은 트리로 해결되지 않습니다. 명제 H4의 반례도 트리 추상화였고, Leduc에서 정방향만 쓰고 트리를 유지하면 2.93에서 멈췄습니다. 역방향 분할이 있으면 트리 여부와 상관없이 21개 모두 정지 상태에 도달했습니다.

그래서 직전 bucket을 기억하는 것의 역할은 추상 게임을 완전 기억으로 만드는 데 있다기보다, 정보가 합쳐진 지점을 찾아 역방향 분할을 가능하게 하는 데 있다고 봅니다.

D. 7-stud: 부분 구현과 완성형 시범을 나눠 본다

질문 D · 실제 크기의 게임에서 고정 추상화(k-means)와 비교하면?

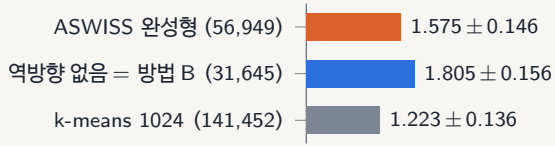
100M roots · 역방향 없는 부분 구현



방법 B − k-means 1024 = $-\mathbf{0.22}$

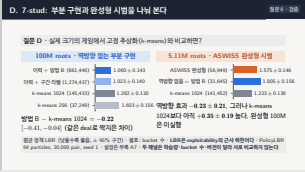
$[-0.41, -0.04]$ (같은 deal로 짝지은 차이)

5.11M roots · ASWISS 완성형 시범



역방향 효과 $-\mathbf{0.23} \pm \mathbf{0.21}$. 그러나 k-means 1024보다 아직 $+\mathbf{0.35} \pm \mathbf{0.19}$ 높다. 완성형 100M은 미실행

평균 정책 LBR (낮을수록 좋음, ± 95% 구간) · 괄호: bucket 수 · **LBR은 exploitability의 근사 하한이다** · PolicyLBR 64 particles, 20,000 pair, seed 1 · 설정은 부록 A7 · **두 패널은 학습량·bucket 수·버전이 달라 서로 비교하지 않는다**

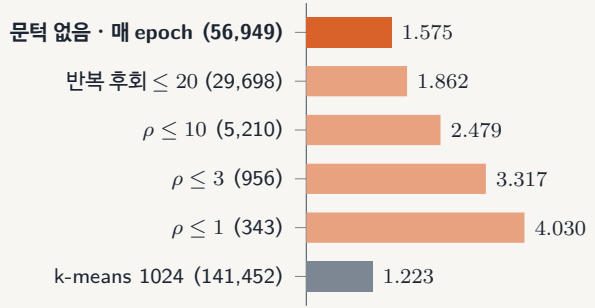


네 번째 질문은 실제 크기의 7-stud입니다. 이 게임은 exploitability를 정확히 잴 수 없어서 LBR로 비교합니다. LBR은 exploitability의 근사 하한이고 낮을수록 좋습니다. 같은 deal, 64 particles, 20,000 pair로 평가했고 seed는 하나입니다.
두 결과를 반드시 나눠서 봐야 합니다. 왼쪽은 1억 roots까지 돌린 실행인데, 이 버전에는 역방향 분할이 없습니다. bucket 이력과 방법 B 라벨을 쓴 비교군이 1.060으로, k-means 1024의 1.282보다 낮았고, 같은 deal로 짝지은 차이는 -0.22, 95% 구간이 -0.41에서 -0.04로 0을 포함하지 않았습니다. 다만 bucket 수가 86만 대 14.5만으로 다릅니다.
오른쪽은 역방향까지 넣은 완성형의 511만 roots 시범입니다. 역방향 분할은 1.805를 1.575로 낮췄지만, 같은 학습량의 k-means 1024의 1.223보다는 아직 0.35 높습니다. 그래서 완성형 ASWISS가 k-means보다 낫다고 말할 수 없습니다. 완성형의 1억 roots 실행은 아직 하지 않았습니다.

두 패널은 학습량, bucket 수, 알고리즘 버전이 모두 다르므로 서로 비교하지 않습니다. 패널 안에서도 bucket 수가 다르다는 점을 함께 봐야 합니다. 비율로 말하면 왼쪽 방법 B는 k-means 1024의 0.83배, 오른쪽 완성형은 1.29배입니다.

E. 판정 문턱: 엄격할수록 나빠졌다

질문 E · 학습기 조건을 흉내 내어, 후회가 작아질 때까지 판정을 미루면 좋아지는가?



7-stud 5.11M roots · 평균 정책 LBR, 20,000 pair (낮을수록 좋음) · 괄호: 마지막 bucket 수

엄격할수록 나빴다

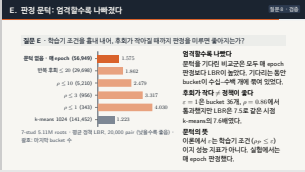
문턱을 기다린 비교군은 모두 매 epoch 판정보다 LBR이 높았다. 기다리는 동안 bucket이 수십–수백 개에 묶여 있었다.

후회가 작다 $\neq$ 정책이 좋다

$\varepsilon = 1$은 bucket 36개, $\rho = 0.86$에서 통과했지만 LBR은 7.5로 같은 시점 k-means의 7.6배였다.

문턱의 뜻

이론에서 ε는 학습기 조건($\rho_{\mathcal{P}} \leq \varepsilon$)이지 성능 지표가 아니다. 실험에서는 매 epoch 판정했다.



마지막 질문은 판정 문턱입니다. 이론의 정리 H5는 학습기가 bucket 후회를 ε 아래로 만든 뒤에 판정한다고 가정합니다. 그래서 7-stud에서 이 조건을 문턱으로 걸고, 후회가 문턱 아래로 내려올 때까지 판정을 미뤄 보았습니다.

결과는 반대였습니다. 주황색 막대가 문턱 없이 매 epoch 판정한 ASWISS로 1.575이고, 문턱을 엄격하게 걸수록 2.479, 3.317, 4.030으로 나빠졌습니다. 기다리는 동안 bucket이 수십에서 수백 개에 묶여 있었기 때문입니다.

또 하나 중요한 관찰은 후회가 작다고 정책이 좋은 것은 아니라는 점입니다. ε 1 비교군은 bucket 36개, 후회 0.86에서 문턱을 통과했지만 LBR은 7.5로 같은 시점 k-means의 7.6배였습니다. 거친 추상화에서는 학습기가 bucket 후회를 줄여도, 인증식의 나머지 두 상쇄 항이 커서 정책이 좋아지지 않습니다.

그래서 ε는 언제 판정할지의 조건, 즉 증명을 위한 학습기 조건이지 성능 지표가 아닙니다. 실험에서는 매 epoch 판정하는 쪽이 이 학습량 안에서 가장 좋았습니다. 이 차이는 이론과 실험 절차 사이에 남은 간격이기도 합니다.

검증된 것과 아직 아닌 것

확인된 것

✓ **증명** · 인증 부등식 (H1), 유한 종료 (H2), 정방향만의 반례 (H4), 표본 판정 (H6), 트리 ⇔ 완전 기억 (H7)

✓ **Kuhn · Leduc** · $\mathcal{P}_0$와 무작위 출발 모두 정지 상태, 추상화 없는 CFR+와 같은 값 (관찰)

✓ **mini-stud** · 역방향 분할로 0.0929 → 0.0277 (seed 3개)

✓ **7-stud 100M** · 역방향 없는 bucket 이력 (방법 B)이 k-means 1024보다 LBR 0.22 낮음 (seed 1)

아직 아닌 것

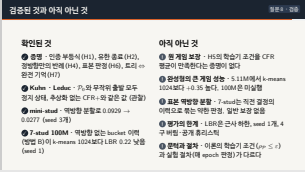
! **원 게임 보장** · H5의 학습기 조건을 CFR 평균이 만족한다는 증명이 없다

! **완성형의 큰 게임 성능** · 5.11M에서 k-means 1024보다 +0.35 높다. 100M은 미실행

! **표본 역방향 분할** · 7-stud는 직전 결정의 이력으로 묶는 약한 판정. 일반 보장 없음

! **평가의 한계** · LBR은 근사 하한, seed 1개, 4구 버림 · 공개 휴리스틱

! **문턱과 절차** · 이론의 학습기 조건 ($\rho_{\mathcal{P}} \le \varepsilon$)과 실험 절차 (매 epoch 판정)가 다르다



검증된 것과 아직 아닌 것을 정직하게 나누겠습니다.

확인된 것입니다. 이론에서는 인증 부등식, 유한 종료, 정방향만으로는 멈추는 반례, 표본 판정, 트리와 완전 기억의 동치가 증명되었습니다. 실험에서는 Kuhn과 Leduc에서 모든 출발점이 정지 상태에 도달해 추상화 없는 CFR+와 같은 값을 냈고, mini-stud에서 역방향 분할이 손실을 3.4배 줄였고, 7-stud 1억 roots에서는 역방향 없는 bucket 이력이 k-means 1024보다 LBR이 낮았습니다.

아직 아닌 것입니다. 첫째, 원 게임 균형의 보장은 학습기 조건에 달려 있는데, 표준 CFR 평균이 그 조건을 만족한다는 증명이 없습니다. 둘째, 역방향까지 넣은 완성형은 큰 게임에서 아직 k-means보다 나쁘고, 1억 roots 실행을 하지 않았습니다. 셋째, 7-stud의 역방향 분할은 정보집합별 표본이 없어서 직전 결정의 이력으로 묶는 약한 판정이고, 일반적인 보장이 없습니다.

넷째, 7-stud의 평가는 LBR이라는 근사 하한이고, seed가 하나이며, 4구의 버림과 공개는 휴리스틱입니다. 다섯째, 이론은 학습기 조건을 만족한 뒤 판정한다고 가정하지만, 실험은 문턱 없이 epoch마다 판정했습니다.

정리와 다음 단계

학습된 전략의 반사실 후회로 나눌 곳을 정하고, **정방향**(bucket 안)과 **역방향**(이력 안) 분할로 추상화가 숨긴 전략적 차이를 되살린다.

요약

- 작은 게임: 어떤 출발점에서도 정지 상태, 추상화 없는 CFR+와 같은 값
- 큰 게임: bucket 이력은 고정 k-means를 넘었다 (역방향 없는 100M). 완성형은 시범 단계
- 이론: 인증과 종료는 증명. 원 게임 균형은 학습기 조건 아래에서만

다음 단계

1. 7-stud에서 역방향 분할을 넣은 ASWISS의 100M 실행
2. 카드 휴리스틱(power 군집) 제거: 카드를 한 장씩 관측으로
3. 학습기 조건을 경로 밖 후회 없이 다시 쓰기 (도달 가중 이득 D)
4. 표본 학습기 MCCFR+의 수렴 보장



정리하겠습니다. ASWISS는 학습된 전략의 반사실 후회로 나눌 곳을 정하고, bucket 안의 차이는 정방향 분할로, 이력 안에 숨은 차이는 역방향 분할로 되살리는 방법입니다.
작은 게임에서는 어떤 출발점에서도 정지 상태에 도달해 추상화 없는 CFR+와 같은 값을 냈습니다. 큰 게임에서는 bucket 이력이라는 아이디어가 고정 k-means를 넘었지만, 그것은 역방향 분할이 없는 버전이었고, 완성형은 아직 시범 단계입니다.
이론에서는 인증과 종료가 증명되었고, 원 게임 균형은 학습기 조건 아래에서만 보장됩니다.
다음 단계는 네 가지입니다. 완성형을 7-stud에서 1억 roots까지 돌리는 것, 카드 쪽 휴리스틱을 없애는 것, 학습기 조건을 CFR이 실제로 만족할 수 있는 형태로 다시 쓰는 것, 그리고 표본 학습기의 수렴 보장입니다.

감사합니다. 자세한 정리와 실험 설정은 부록에 있습니다.

A1. 기호와 정의

반사실 가치 $q^\sigma(I,a) = \sum_{h\in I} \pi^\sigma_{-i}(h) \sum_{z \sqsupseteq ha} \pi^\sigma(ha,z)\, u_i(z)$

순간 후회 $r^\sigma(I,a) = q^\sigma(I,a) - \sum_b \sigma(I,b)\, q^\sigma(I,b), \quad r^\sigma(I,\varnothing) = 0$ ("없음")

모은 후회 $r(K) = \sum_{I\in K} r(I), \quad r(B) = \sum_{K\subseteq B} r(K)$

ψ와 라벨 집합 $\psi(x) = \max(0, \max_a x_a), \quad M(x) = \arg\max_{\ell \in A\cup\{\varnothing\}} x_\ell$. B가 일치: $\bigcap_{K\subseteq B} M(r(K)) \neq \emptyset$

세 양 $\rho_{\mathcal{P}}(\sigma) = \sum_B \psi(r(B))$, $G_B = \sum_{K\subseteq B} \psi(r(K)) - \psi(r(B))$, $H_K = \sum_{I\in K} \psi(r(I)) - \psi(r(K))$ (ψ가 준선형이라 $G, H \geq 0$)

bucket 이력 $\kappa_{\mathcal{P}}(I) = \big(i, A(I), b(I^1), a^1, \ldots, b(I^m), a^m, o(I)\big)$. 7-stud 구현은 직전 bucket만: $(b(I^m), a^m, o(I))$

허용 분할 모든 bucket이 이력류(같은 bucket 이력의 정보집합 묶음)의 합집합. $\mathcal{P}_0$: (플레이어, 행동집합, 자기 결정 순번)으로 묶음

NashConv $\sum_i \big(\max_{\sigma_i'} u_i(\sigma_i', \sigma_{-i}) - u_i(\sigma)\big)$. 실험의 exploitability = NashConv / 2

출처: 증명 초안 ASWISS_HISTORY_PROOF_KO.md §2. G는 완전 기억 유한 2인 영합 전개형 게임, σ는 lifting된 행동 전략($\sigma(I) = \sigma(b(I))$).


A1. 기호와 정의
부록 A1

반사실 가치 $q^{\sigma}(I,a) = \sum_{h \in I} \pi^{\sigma}_{-i}(h) \sum_{(h,a,h') \in \mathcal{H}} \pi^{\sigma}(h,a)\, u_i(h')$

순간 후회 $r^{\sigma}(I,a) = q^{\sigma}(I,a) - \sum_{a'} \sigma(I,a')\, q^{\sigma}(I,a'), \quad r^{\sigma}(I,a) \geq 0$ ("없음")

요준 후회 $r(A) = \sum_{I \in A} r(I), \quad r(B) = \sum_{I \in B} r(A)$

ϵ-약 라벨링 집합 $\psi(r) = \max(0, \max_{\mathcal{C}} w_{\mathcal{C}})$, $M(\sigma) = \arg\max_{(A,a) \in \sigma} w$, B가 일치: $\prod_{C \in \mathcal{B}} M(r(A)) \neq \emptyset$

세영 $\rho^{\sigma}(\sigma') = \sum_{\mathcal{C}} \psi(r(B))$, $Gx = \sum_{C \in \mathcal{G}} \psi(r(K)) - \psi(r(B))$, $Hx = \sum_{C \in \mathcal{H}} \psi(r(I)) - \psi(r(K))$ (σ가 준선형이라 $G, H \geq 0$)

bucket 이득 $\kappa_{\mathcal{B}}(I) = \{A, A(\mathcal{B}), b(\mathcal{F}), a^1, \ldots, b(\mathcal{F}^n), a^n, a(\mathcal{B})\}$, 7-stud 구현은 직전 bucket만: $(b(\mathcal{F}^n), a^n, a(\mathcal{B}))$

적응 분할 모든 bucket이 이득률 (같은 bucket 이력의 정보집합 등장)의 합집합. $\mathcal{P}_0$: (플레이어, 행동집합, 차기 결정 순번) 요로 묶음

NashConv $\sum_i (\max_{\sigma'_i} u_i(\sigma'_i, \sigma_{-i}) - u_i(\sigma))$, **실험의 exploitability** $= \text{NashConv} \;/\; 2$

출처: 공평 추천 ASWISS_HISTORY_PROOF_KO.md §2. 이는 원전 기억 위한 2인 영합 전개형 게임, r는 lifting된 행동 간략($r(I) = r(K(I))$).

본문에서 말로 설명한 양들의 정확한 정의입니다. 라벨 판정과 인증식이 모두 이 정의 위에 서 있습니다. psi는 후회 벡터에서 "바꿔서 얻을 수 있는 최대 이득"이고, 이것이 준선형이기 때문에 두 상쇄 항 G와 H가 항상 0 이상입니다. 실험에서 말한 exploitability는 NashConv의 절반입니다.

A2. 정리 H1 (인증)과 정리 H2 (유한 종료)

보조정리 H5 (NashConv 분해) · β_i를 σ_{-i}에 대한 순수 최선 응답이라 하면

$$\mathrm{NashConv}(\sigma) = \sum_i \sum_{I \in \mathcal{I}_i} \pi_i^{\beta_i}(I)\, r^{\sigma}(I, \beta_i(I)) \le \sum_I \psi(r^{\sigma}(I)).$$

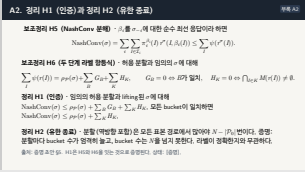
보조정리 H6 (두 단계 라벨 항등식) · 허용 분할과 임의의 σ에 대해

$$\sum_I \psi(r(I)) = \rho_{\mathcal{P}}(\sigma) + \sum_B G_B + \sum_K H_K, \qquad G_B = 0 \Leftrightarrow B\text{가 일치}, \quad H_K = 0 \Leftrightarrow \bigcap_{I \in K} M(r(I)) \neq \emptyset.$$

정리 H1 (인증) · 임의의 허용 분할과 lifting된 σ에 대해
$\mathrm{NashConv}(\sigma) \le \rho_{\mathcal{P}}(\sigma) + \sum_B G_B + \sum_K H_K$. 모든 bucket이 일치하면
$\mathrm{NashConv}(\sigma) \le \rho_{\mathcal{P}}(\sigma) + \sum_K H_K$.

정리 H2 (유한 종료) · 분할(역방향 포함)은 모든 표본 경로에서 많아야 $N - |\mathcal{P}_0|$번이다. 증명: 분할마다 bucket 수가 엄격히 늘고, bucket 수는 N을 넘지 못한다. 라벨이 정확한지와 무관하다.

출처: 증명 초안 §5. H1은 H5와 H6을 잇는 것으로 증명된다. 상태: [증명].


A2. 정리 H1 (인증)과 정리 H2 (유한 종료) 부록 A2

보조정리 H5 (NashConv 분해) · β_i를 σ_{-i}에 대한 순수 최선 응답이라 하면

$$\mathrm{NashConv}(\sigma) = \sum_{\sigma'} \sum_{I \in \mathcal{I}_i^+} \pi_i^{\sigma'}(I)\, r^{\sigma}(I, \beta_i(I)) \le \sum_{I} v(r^{\sigma}(I)).$$

보조정리 H6 (두 단계 래핑 항등식) · 허용 분할과 임의의 v에 대해

$$\sum_{J} v(r(I)) = \rho_{\mathcal{P}}(\sigma) + \sum_{G} G_R v \sum_{H} H_R, \qquad G_R = 0 \Leftrightarrow R\text{가 일치}, \quad H_R = 0 \Leftrightarrow \textstyle\bigcap_{G \ni R} M(\sigma(I)) \neq \emptyset.$$

정리 H1 (인증) · 임의의 허용 분할과 lifting점 σ에 대해
$\mathrm{NashConv}(\sigma) \le \rho_{\mathcal{P}}(\sigma) + \sum_R G_R + \sum_R H_R$. 모든 bucket이 일치하면
$\mathrm{NashConv}(\sigma) \le \rho_{\mathcal{P}}(\sigma) + \sum_R H_R$.

정리 H2 (유한 종료) · 분할(역방향 포함)은 모든 표본 경로에서 많아야 $N - |\mathcal{P}_0|$번이다. 증명: 분할마다 bucket 수가 엄격히 늘고, bucket 수는 N을 넘지 못한다. 라벨이 정확한지와 무관하다.

출처: 공정 포커 문제 — 비슷한 비교를 하는 것으로 공정한다. 상태 그림.

인증식이 어떻게 나오는지의 골격입니다. 먼저 NashConv를 최선 응답 경로 위의 정보집합별 후회의 합으로 쓰고, 그 합을 bucket 후회와 두 상쇄 항으로 정확히 나눕니다. 그래서 부등식은 첫 단계에서만 생깁니다. 유한 종료는 분할이 bucket을 쪼개기만 하고 정보집합을 옮기지 않는다는 데서 바로 나옵니다.

A3. 조건부 결과와 학습기 조건

ε-오라클 가정 · 매 라운드 학습기가 $\rho_{\mathcal{P}}(\bar{\sigma}) \le \varepsilon$인 lifting된 프로필을 낸다. 라벨은 정확히 계산한다.

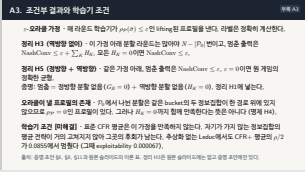
정리 H3 (역방향 없이) · 이 가정 아래 분할 라운드는 많아야 $N - |\mathcal{P}_0|$번이고, 멈춘 출력은 $\mathrm{NashConv} \le \varepsilon + \sum_K H_K$. 모든 $H_K = 0$이면 $\mathrm{NashConv} \le \varepsilon$.

정리 H5 (정방향 + 역방향) · 같은 가정 아래, 멈춘 출력은 $\mathrm{NashConv} \le \varepsilon$. $\varepsilon = 0$이면 원 게임의 정확한 균형.
증명: 멈춤 = 정방향 분할 없음($G_B = 0$) + 역방향 분할 없음($H_K = 0$). 정리 H1에 넣는다.

오라클이 낼 프로필의 존재 · $\mathcal{P}_0$에서 나뉜 분할은 같은 bucket의 두 정보집합이 한 경로 위에 있지 않으므로 $\rho_{\mathcal{P}} = 0$인 프로필이 있다. 그러나 $H_K = 0$까지 함께 만족한다는 뜻은 아니다 (명제 H4).

학습기 조건 [미해결] · 표준 CFR 평균은 이 가정을 만족하지 않는다. 자기가 가지 않는 정보집합의 평균 전략이 거의 고쳐지지 않아 그곳의 후회가 남는다. 추상화 없는 Leduc에서도 CFR+ 평균의 $\rho/2$가 0.0855에서 멈췄다 (그때 exploitability 0.000067).

출처: 증명 초안 §6, §8, §11과 원본 슬라이드의 이론 표. 정리 H3은 원본 슬라이드에는 없고 증명 초안에만 있다.


A3. 조건부 결과와 학습기 조건
부록 A3
ε-오라클 가정 · 매 라운드 학습기가 $\rho_{BV}(\sigma) \le \varepsilon$인 lifting된 프로필을 낸다. 라벨은 정확히 계산한다.
정리 H3 (역방향 없이) · 이 가정 아래 분할 라운드는 많아야 $N \cdot |P_1|$번이고, 멈춤 출력은 $\mathrm{NashConv} \le \varepsilon + \sum_a H_a$. 모든 $H_a = 0$이면 $\mathrm{NashConv} \le \varepsilon$.
정리 H5 (정방향 + 역방향) · 같은 가정 아래, 멈춤 출력은 $\mathrm{NashConv} \le \varepsilon$, $\varepsilon = 0$이면 둔 게임의 정확한 균형.
증명 · 멈춤 = 정방향 분할 없음 ($G_a = 0$) + 역방향 분할 없음 ($H_a = 0$). 정리 H1에 넣는다.
오라클이 낼 프로필의 존재 · P_1에서 나뉜 분할은 같은 bucket의 두 정보집합이 한 경로 위에 있지 않으므로 $\rho_{BV} = 0$인 프로필이 있다. 그러나 $H_a = 0$까지 힘써 만족한다는 뜻은 아니다 (명제 H4).
학습기 조건 [미해결] · 표준 CFR 평균은 이 가정을 만족하지 않는다. 저기가 가지 않는 정보집합의 평균 전략이 거의 고려되지 않아 그곳의 후회가 남는다. 추상화 없는 Leduc에서도 CFR+ 평균의 $\rho/2$가 0.0855에서 멈췄다 (그래 exploitability 0.000067).
출처: 증명 초안 §4, §A. §3.1 과 원본 슬라이드의 이론 표. 정리 H1은 원본 슬라이드에는 없고 증명 초안에만 있다.

원 게임에 대한 보장이 정확히 어떤 가정 위에 있는지 적었습니다. 핵심은 ε-오라클, 즉 학습기가 bucket 후회를 ε 아래로 만든다는 가정입니다. 역방향이 없으면 이력 안 상쇄가 남을 수 있고, 역방향이 있으면 그것까지 0이 되어 NashConv ε 이하가 됩니다. 그러나 표준 CFR 평균은 이 가정을 만족하지 않으므로, 이 결과를 그대로 CFR에 적용할 수는 없습니다. 이것이 가장 큰 미해결 문제입니다.

A4. 합쳐진 지점, 역방향 규칙, 트리 구조

보조정리 H3 (정보가 합쳐진 지점) · $I \neq I'$ 이고 $\kappa(I) = \kappa(I')$ 이면 어떤 $j \leq m$에서

$$I^j \neq I'^j, \qquad \kappa(I^j) \neq \kappa(I'^j), \qquad b(I^j) = b(I'^j).$$

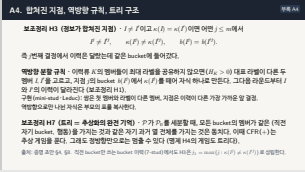
즉 j번째 결정에서 이력은 달랐는데 같은 bucket에 들어갔다.

역방향 분할 규칙 · 이력류 K의 멤버들이 최대 라벨을 공유하지 않으면 ($H_K > 0$) 대표 라벨이 다른 두 멤버 I, I'을 고르고, 지점 j의 bucket $b(I^j)$에서 $\kappa(I^j)$를 떼어 자식 하나로 만든다. 그다음 라운드부터 I와 I'의 이력이 달라진다 (보조정리 H1).

구현 (mini-stud · Leduc): 쌍은 첫 멤버와 라벨이 다른 멤버, 지점은 이력이 다른 가장 가까운 앞 결정.
역방향으로만 나뉜 자식은 부모의 표를 복사한다.

보조정리 H7 (트리 = 추상화의 완전 기억) · $\mathcal{P}$가 $\mathcal{P}_0$를 세분할 때, 모든 bucket의 멤버가 같은 (직전 자기 bucket, 행동)을 가지는 것과 같은 자기 과거 열 전체를 가지는 것은 동치다. 이때 CFR(+)는 추상 게임을 푼다. 그래도 정방향만으로는 멈출 수 있다 (명제 H4의 게임도 트리다).

출처: 증명 초안 §4, §8. 직전 bucket만 쓰는 bucket 이력 (7-stud)에서도 H3은 $j_1 = \max\{j : \kappa(I^j) \neq \kappa(I'^j)\}$로 성립한다.


A4. 합쳐진 지점, 역방향 규칙, 트리 구조 부록 A4

보조정리 H3 (정보가 합쳐진 지점) · $I \neq I'$이고 $\kappa(I) = \kappa(I')$이면 어떤 $j \leq m$에서

$$F \neq F', \quad \kappa(F) \neq \kappa(F'), \quad h(F) = h(F').$$

즉 J번째 결정에서 이력은 달랐는데 같은 bucket에 들어갔다.

역방향 분할 규칙 · 이력류 K의 멤버들이 최대 라벨을 공유하지 않으면($U_K > 0$) 대표 라벨이 다른 두 멤버 L, L'을 고르고, 지점 J의 bucket $h(F)$에서 $\kappa(F)$를 택해 거식 하나씩 만든다. 그다음 라운드부터 I와 F의 이력이 달라진다 (보조정리 H1).
구현(mini-mud 4.ebuc): 방은 첫 멤버와 라벨이 다른 멤버, 지점은 어떡이 다른 거장 가까운 잘 결정.
역방향으로만 나눠 자식은 부모의 표를 똑같이 투표한다.

보조정리 H7 (트리 = 추상화의 완전 기억) · 구 가 T_1를 세분할 때, 모든 bucket의 멤버가 같은 (직전 자기 bucket, 행동)을 가지는 것과 같은 자기 과거 열 건체를 가지는 것은 동치다. 이때 CFR$(+)$는 수상 게임을 푼다. 그래도 장방향만으로는 점을 수 있다 (멘게 H4의 게임도 트리다).

출처: 공업 프은실, 당4. 직전 bucket은 프는 bucket 이력(구-mud 학식도 비슷는 $\kappa = \max\{j : \kappa(F) \neq \kappa(F')\}$로 상일한다.

역방향 분할이 가능한 이유는 보조정리 H3입니다. 같은 이력에 들어간 두 정보집합은 과거 어딘가에서 이력은 달랐는데 같은 bucket에 들어간 지점이 반드시 있습니다. 그 지점의 bucket을 나누면 이후 두 정보집합의 이력이 갈라집니다. 트리 구조와 완전 기억의 동치인 H7도 여기 적었습니다. 트리는 학습기의 수렴을 돕지만 분할의 수렴을 보장하지는 않습니다.

A5. 표본 판정 (정리 H6)과 구현의 차이

설정 · 라운드 k에서 $\bar{\sigma}$을 고정하고, 두 좌석 모두 $\bar{\sigma}$로 두는 외부 샘플링 순회를 n_k번 한다. 순회마다 방문한 정보집합의 표본 후회를 이력별로 더해 n_k로 나눈 것이 $\hat{r}(K)$.

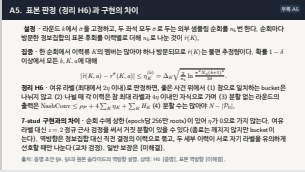
집중 · 한 순회에서 이력류 K의 멤버는 많아야 하나 방문되므로 $\hat{r}(K)$는 불편 추정량이다. 확률 $1-\delta$ 이상에서 모든 k, K, a에 대해

$$|\hat{r}(K,a) - r^{\bar{\sigma}}(K,a)| \le \eta_K^{(k)} = \Delta_K \sqrt{\frac{2}{n_k} \ln \frac{\pi^2 N_A (k+1)^2}{3\delta}}.$$

정리 H6 · 여유 라벨(최대에서 2η 이내)로 판정하면, 좋은 사건 위에서 (1) 참으로 일치하는 bucket은 나뉘지 않고 (2) 나뉠 때 각 이력은 참 최대 라벨과 4η 이내인 자식으로 가며 (3) 분할 없는 라운드의 출력은 $\mathrm{NashConv} \le \rho_{\mathcal{P}} + 4\sum_K \eta_K + \sum_K H_K$ (4) 분할 수는 많아야 $N - |\mathcal{P}_0|$.

7-stud 구현과의 차이 · 순회 수에 상한(epoch당 256만 roots)이 있어 η가 0으로 가지 않는다. 여유 라벨 대신 $z = 2$ 정규 근사 검정을 써서 거짓 분할이 있을 수 있다(종료는 깨지지 않지만 bucket이 는다). 역방향은 정보집합 대신 직전 결정의 이력으로 묶고, 두 세부 이력이 서로 자기 라벨을 유의하게 선호할 때만 나눈다(교차 검정). 일반 보장은 [미해결].

출처: 증명 초안 §9, §8과 원본 슬라이드의 역방향 설명. 상태: H6 [증명], 표본 역방향 [미해결].


A5. 표본 판정 (정리 H6)과 구현의 차이
부록 A5

설정 · 라운드 k에서 s를 고정하고, 두 좌석 모두 s로 두는 외부 샘플링 순회를 n_k번 한다. 순회마다 방문한 정보집합의 표본 투영을 이벤트로 다룸 — n_k는 논는 것이 $I(K)$.

질문 · 한 순회에서 이벤트 K의 명비는 많아야 하나 방문되므로 $I(K)$는 몰면 추정량이다. 확률 $1-\delta$ 이상에서 모든 k, K, s에 대해

$$|\hat{\imath}(K,n) - i^{\pi}(K,s)| \le \varepsilon_{k,\delta}^{(k)} = \Delta_k \sqrt{\frac{n}{2}} \ln \frac{2|\mathcal{K}_k|(n+1)}{\delta}$$

정리 H6 · 여유 라벨(최대에서 2_0 아래)로 판정하면, 좋은 사건 위에서 (1) 참으로 일치하는 bucket은 나눠지 않고 (2) 나빴 때 각 이벤은 분 최대 라벨과 4_0 아내인 거이므로 가며 (3) 분할 없는 라운드의 출력은 $\mathrm{NashConv} \le \rho_{\mathcal{P}} \cdot 4\sum_k \eta_k + \sum_{\mathcal{U}} R_{\mathcal{U}}$ (4) 분할 수는 많아야 $N - |\mathcal{P}_0|$.

7-stud 구현과의 차이 · 순회 수에 상한 (epoch당 256만 roots)이 있어 η_k가 0으로 가지 않는다. 여유 라벨 대신 $z = 2$ 정규 근사 검정을 써서 거짓 분할이 있을 수 있다 (종료는 깨지지 않지만 bucket이 는다). 역방향 정보집합 대신 직전 결정의 이력으로 묶고, 두 세부 이벤이 서로 자기 라벨을 유의하게 선호할 때만 나눈다 (교차 검정). 일반 보장은 미해결.

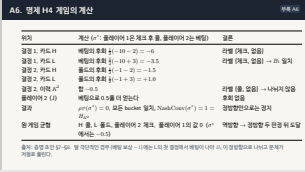
출처: 유명 조언 당, 당과 원본 슬라이드의 '역방향 설명. 상태: 1M. [설명], 표준 '역방향' [미해결].

큰 게임에서는 라벨을 정확히 계산할 수 없어 표본으로 추정합니다. 이론적으로는 순회 수를 늘리면 추정 오차 eta가 0으로 가고, 인증식에 4 시그마 eta가 더해질 뿐입니다. 하지만 실제 7-stud 구현은 순회 수에 상한이 있고, 여유 라벨 대신 정규 근사 검정을 쓰며, 역방향 분할은 정보집합 대신 직전 결정의 이력으로 묶는 약한 판정을 씁니다. 이 부분은 보장이 없는 실험적 선택입니다.

A6. 명제 H4 게임의 계산

위치	계산 (σ^*: 플레이어 1은 체크 후 콜, 플레이어 2는 베팅)	결론
결정 1, 카드 H	베팅의 후회 $\frac{1}{2}(-10-2)=-6$	라벨 {체크, 없음}
결정 1, 카드 L	베팅의 후회 $\frac{1}{2}(-10+3)=-3.5$	라벨 {체크, 없음} $\to B_1$ 일치
결정 2, 카드 H	폴드의 후회 $\frac{1}{2}(-1-2)=-1.5$	
결정 2, 카드 L	폴드의 후회 $\frac{1}{2}(-1+3)=+1.0$	
결정 2, 이력 K^2	합 -0.5	라벨 {콜, 없음} $\to$ 나뉘지 않음
플레이어 2 (J)	베팅으로 0.5를 더 얻는다	후회 없음
결과	$\rho_{\mathcal{P}}(\sigma^*)=0$, 모든 bucket 일치, $\mathrm{NashConv}(\sigma^*)=1=H_{K^2}$	정방향만으로는 정지
원 게임 균형	H 콜, L 폴드, 플레이어 2 체크. 플레이어 1의 값 0 (σ^*에서는 -0.5)	역방향 $\to$ 정방향 두 판정 뒤 도달

출처: 증명 초안 §7–§8. 덜 극단적인 경우(베팅 보상 -1)에는 L의 첫 결정에서 베팅이 나아 B_1이 정방향으로 나뉘고 문제가 저절로 풀린다.



A6. 명제 H4 게임의 계산

부록 A6

위치	계산 (σ^*: 플레이어 1은 체크 후 콜, 플레이어 2는 베팅)	결론
결정 1, 카드 H	베팅의 후회 $\frac{1}{2}(-10-2) = -6$	라벨 [체크, 압倒]
결정 1, 카드 L	베팅의 후회 $\frac{1}{2}(-10+3) = -3.5$	라벨 [체크, 압倒] → B_1 일치
결정 2, 카드 H	콜드의 후회 $\frac{1}{2}(-1-2) = -1.5$	
결정 2, 카드 L	콜드의 후회 $\frac{1}{2}(-1+3) = +1.0$	
결정 2, 이력 A^2	합 -0.5	라벨 [콜, 압倒] → 나머지 압倒
플레이어 2 (J) 결과	베팅으로 0.5를 더 얻는다	후회 없음
	$\mu\pi(\sigma^*) = 0$, 모든 bucket 일치, $\mathrm{NashConv}(\sigma^*) = 1 = H_{\sigma^*}$	경험합인으로는 정지
왕 게임 균형	H 콜, L 폴드, 플레이어 2 체크. 플레이어 1의 값 0 (σ^*에서는 -0.5)	역방향 → 경험합 두 판권 위 도달

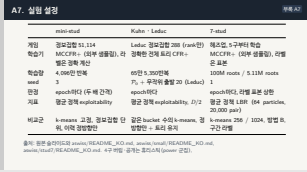
출처: 공명 추천 답 설. 및 여러적인 연주 (베팅 보상 ·) 에는 L의 첫 경험에서 베팅이 나타 B_1 이 경험합으로 나눠고 문제가 지정로 돌린다.

본문의 예시 게임을 숫자로 끝까지 계산한 표입니다. 첫 결정에서는 두 카드 모두 체크가 압도적이라 나뉠 이유가 없고, 두 번째 결정에서는 H와 L이 같은 이력이라 L의 양의 후회가 상쇄됩니다. 그 결과 bucket 후회가 0이고 모든 bucket이 일치하는데 NashConv는 1입니다. 남은 오차는 정확히 이력 안 상쇄와 같습니다.

A7. 실험 설정

	mini-stud	Kuhn · Leduc	7-stud
게임	정보집합 51,114	Leduc 정보집합 288 (rank만)	헤즈업, 5구부터 학습
학습기	MCCFR+ (외부 샘플링), 라벨은 정확 계산	정확한 전체 트리 CFR+	MCCFR+ (외부 샘플링), 라벨은 표본
학습량	4,096만 반복	65만 5,350반복	100M roots / 5.11M roots
seed	3	$\mathcal{P}_0$ + 무작위 출발 20 (Leduc)	1
판정	epoch마다 (두 배 간격)	epoch마다	epoch마다, 라벨 표본 상한
지표	평균 정책 exploitability	평균 정책 exploitability, $D/2$	평균 정책 LBR (64 particles, 20,000 pair)
비교군	k-means 고정, 정보집합 단위, 이력 정방향만	같은 bucket 수의 k-means, 정방향만 + 트리 유지	k-means 256 / 1024, 방법 B, 구간 라벨

출처: 원본 슬라이드와 aswiss/README_KO.md, aswiss/small/README_KO.md, aswiss/stud7/README_KO.md. 4구 버림·공개는 휴리스틱(power 군집).


A7. 실험 설정

부록 A7

	mini-stud	Kuhn · Leduc	7-stud
게임	정보집합 51, 114	Leduc 정보집합 288 (rank만)	해즈엔, 5구부터 학습
학습기	MCCFR+, (외부 샘플링), 라벨은 정확 계산	정확한 전체 트리 CFR+	MCCFR+, (외부 샘플링), 라벨은 표본
학습량	4,096만 반복	6만만 5,350만반복	100M roots / 5.11M roots
seed	3	T_0 + 무작위 출발 20 (Leduc)	1
만정	epoch마다 (두 배 간격)	epoch마다	epoch마다, 라벨 표본 상한
지표	평균 정책 exploitability	평균 정책 exploitability, $E_1/2$	평균 정책 LBR (64 particles, 20,000 pair)
비교군	k-means 고정, 정보집합 단위, 이적 정확합만	같은 bucket 수의 k-means, 정확합만 + 트리 유지	k-means 256 / 1024, 랭법 8, 구간 라벨

출처: 원본 슬라이드의 assiss/README_KO.md, assiss/result/README_KO.md, assiss/stud7/README_KO.md. ※구하입: 공개는 표준스퀴(power 군집).

실험별 설정을 한 표에 모았습니다. 작은 게임은 정확한 CFR+와 정확한 exploitability, 7-stud는 표본 학습기와 LBR이라는 점이 가장 큰 차이입니다. 7-stud는 seed가 하나라는 점도 한계로 함께 기억해 주십시오.

A8. 비교 조건의 차이 (같은 조건의 비교가 아닌 것)

mini-stud 비교군 (4,096만 반복)	bucket 수 (seed 1/2/3)
정보집합 단위, 매 epoch	12,683 / 12,839 / 12,888
bucket 이력, 정방향만	12,895 / 13,132 / 13,158
bucket 이력 + 역방향 (ASWISS)	15,745 / 16,304 / 15,811
카드 k-means 고정	7,051

mini-stud의 대가: 역방향 분할 뒤 $H_K = 0$이 되었을 때 bucket 이력이 정보집합의 99.4–99.8%를 구별했다. 이 게임에서 $H_K = 0$을 얻으려면 이력이 사실상 정보집합을 다 기억해야 했다. 큰 게임에서도 그런지는 미해결.

7-stud에서 다른 것

- 100M과 5.11M은 학습량과 버전(역방향 유무)이 다르다
- 같은 패널 안에서도 bucket 수가 다르다 (예: 86만 대 14.5만)
- 5.11M에서는 k-means 256도 측정했지만 여기서는 더 강한 k-means 1024만 보였다

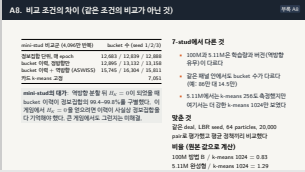
맞춘 것

같은 deal, LBR seed, 64 particles, 20,000 pair로 평가했고 평균 정책끼리 비교했다

비율 (원본 값으로 계산)

100M 방법 B / k-means 1024 = 0.83

5.11M 완성형 / k-means 1024 = 1.29



공정한 비교를 위해 조건이 다른 부분을 모았습니다. mini-stud에서는 비교군마다 최종 bucket 수가 다르고, ASWISS가 가장 많은 bucket을 썼습니다. 또 역방향 분할로 이력 안 상쇄를 0으로 만들었을 때 bucket 이력이 정보집합을 거의 다 구별했다는 점은 이 방법의 대가로 기억해야 합니다. 7-stud에서는 두 실행의 학습량과 버전이 다르고, 같은 실행 안에서도 bucket 수가 다릅니다. 평가 조건은 같은 deal과 같은 LBR 설정으로 맞췄습니다.

A9. 판정 문턱 실험의 세부

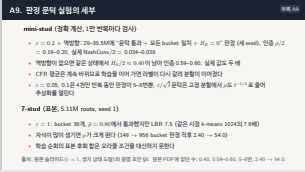
mini-stud (정확 계산, 1만 반복마다 검사)

- $\varepsilon = 0.2$ + 역방향: 29–30.5M에 "문턱 통과 + 모든 bucket 일치 + $H_K = 0$" 판정 (세 seed). 인증 $\rho/2$ = 0.19–0.20, 실제 NashConv/2 = 0.034–0.038
- 역방향이 없으면 같은 상태에서 $H_K/2 \approx 0.40$이 남아 인증 0.59–0.60, 실제 값도 두 배
- CFR 평균은 계속 바뀌므로 학습을 이어 가면 라벨이 다시 갈려 분할이 이어졌다
- $\varepsilon =$ 0.05, 0.1은 4천만 반복 동안 판정이 5–8번뿐. $\varepsilon/\sqrt{t}$ 문턱은 고정 분할에서 ρ도 $t^{-1/2}$로 줄어 추상화를 얼린다

7-stud (표본, 5.11M roots, seed 1)

- $\varepsilon = 1$: bucket 36개, $\hat{\rho} = 0.86$에서 통과했지만 LBR 7.5 (같은 시점 k-means 1024의 7.6배)
- 자식이 많이 생기면 ρ가 크게 뛴다 (149 → 956 bucket 판정 직후 2.40 → 54.0)
- 학습 순회의 표본 후회 합은 오라클 조건을 대신하지 못한다

출처: 원본 슬라이드($\varepsilon = 1$, 정지 상태 도달)와 증명 초안 §6. 원본 PDF에 없던 수: 0.40, 0.59–0.60, 5–8번, 2.40 → 54.0.



본문의 문턱 실험을 더 자세히 적었습니다. mini-stud에서는 문턱을 ε 0.2로 두었을 때 이론의 정지 상태가 실제로 나왔고, 실제 값이 인증값 아래에 있었습니다. 그러나 문턱이 작으면 거의 판정하지 못했고, 시간에 따라 줄어드는 문턱은 추상화를 얼렸습니다. 7-stud에서는 문턱이 정책을 오히려 나쁘게 했습니다.

A10. 원본 이후 작업 (참고): 라벨용 혼합 평균

원본 PDF에 없는 내용이다. 발표 본문에서는 쓰지 않았다.

문제 · 평균 전략은 자기가 거의 가지 않게 된 곳에서 초반 값에 멈춘다. 정책에는 영향이 없지만 (실현 계획의 질량이 0), 자주 가는 정보집합의 라벨이 그 아래의 멈춘 값으로 계산되어 "그 행동이 사실 더 좋다"는 분할 신호를 놓친다.

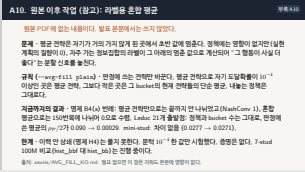
규칙 (`--avg-fill plain`) · 판정에 쓰는 전략만 바꾼다. 평균 전략으로 자기 도달확률이 10^{-4} 이상인 곳은 평균 전략, 그보다 작은 곳은 그 bucket의 현재 전략들의 단순 평균. 내놓는 정책은 그대로다.

지금까지의 결과 · 명제 B4(a) 반례: 평균 전략만으로는 끝까지 안 나뉘었고 (NashConv 1), 혼합 평균으로는 150반복에 나뉘어 0으로 수렴. Leduc 21개 출발점: 정책과 bucket 수는 그대로, 판정에 쓴 평균의 $\rho_{\mathcal{P}}/2$가 0.090 $\rightarrow$ 0.00029. mini-stud: **차이 없음** (0.0277 $\rightarrow$ 0.0271).

한계 · 이력 안 상쇄 (명제 H4)는 풀지 못한다. 문턱 10^{-4} 한 값만 시험했다. 증명은 없다. 7-stud 100M 비교 (hist_bbf 대 hist_bb)는 진행 중이다.

출처: aswiss/AVG_FILL_KO.md. 필요 없으면 이 장은 지워도 본문에 영향이 없다.

└A10. 원본 이후 작업 (참고): 라벨용 혼합 평균


A10. 원본 이후 작업 (참고): 라벨용 혼합 평균
부록 A10
원본 PDF에 없는 내용이다. 발표 본문에서는 소개 않았다.
문제 · 평균 견적은 자기가 가진 가지 않게 된 곳에서 조반 없에 멈춘다. 정책에는 영향이 없지만 (실현 계획의 결정이 아), 저주 가는 정보없었의 라벨이 그 아래의 항초 값으로 계산되어 "그 항동이 사실 더 줄다"는 분할 신호를 놓친다.
규칙 (--avg-fill plain) · 한정에 쓰는 견적만 바꾼다. 평균 견적으로 자기 도달확률이 10^{-4} 이상인 곳은 평균 견적, 그보다 작은 곳은 그 bucket의 현재 견적들의 단순 평균. 내놓는 정책은 그대로다.
저금까지의 결과 · 명제 84(a) 반례: 평균 견적만으로는 끝까지 안 나뉘었고(NashConv 1), 혼합 평균으로는 150번째에 나뉘어 0으로 수렴. Leduc 21개 출발점: 정책과 bucket 수는 그대로, 만장에은 평균의 p_{fill}/2가 0.090 → 0.00029. mini-stud: 차이 없음 (0.0277 → 0.0271).
한계 · 이력 안 상세 (명제 H4) 는 좋지 못한다. 문턱 10^{-4} 한 값만 시험했다. 경쟁은 없다. 7-stud 100M 비교(bot_bbf 대 bot_bb)는 진행 중이다.
출처: assets/AVG_FILL_KO.md. 발표 없으면 이 경은 자체도 본문에 영향이 없다.

질의응답에 대비한 참고 장입니다. 원본 발표 이후에 한 작업으로, 학습기 조건 문제 중 경로 밖 부분을 라벨 계산에서만 우회하는 방법입니다. 내놓는 정책은 바꾸지 않습니다. 작은 게임에서는 경로 밖 라벨 때문에 멈췄던 실행이 풀렸고, 7-stud에서의 효과는 1억 roots 실행으로 확인하는 중입니다.